

BEAGLEBONE AI ROBOT

System Description, ROS Architecture, and
Navigation and Mapping Algorithms

Jason Stumfol

Repository: <https://github.com/jstumfol12/bbai-robot>

State of the robot as of 2 October 2026

*Sections marked **In flux** describe configuration that was being changed while this document was written. The repository is the authority for anything that disagrees with this text.*

Contents

List of Figures	5
List of Tables	6
1 SYSTEM OVERVIEW	8
1.1 PURPOSE AND SCOPE	8
1.2 GOALS	8
1.3 PLATFORM SUMMARY	9
1.4 DIVISION OF WORK BETWEEN THE COMPUTERS	9
1.5 STATUS	10
2 HARDWARE REFERENCE	12
2.1 CHASSIS AND DRIVETRAIN	12
2.2 BEAGLEBONE AI	12
2.3 ROBOTICS CAPE	13
2.3.1 Power	13
2.3.2 Motor drivers	14
2.3.3 Encoder ports	15
2.3.4 Sensors on the cape	15
2.3.5 Connectors	15
2.4 SENSORS	16
2.4.1 MPU-9250 IMU	16
2.4.2 RPLIDAR A1M8	16
2.4.3 EM-506 GPS	16
2.4.4 Wheel encoders (A1230)	16
2.4.5 OAK-D-Lite camera	16
2.4.6 PS4 controller	17
2.5 POWER BUDGET	17
3 HARDWARE BRING-UP	18
3.1 OPERATING SYSTEM AND BOOT	18

3.2	DEVICE TREE	18
3.3	SERIAL PORTS	19
3.4	LIBROBOTCONTROL FORK	20
3.5	MOTOR 1 DIRECTION FIRMWARE ON PRU2_0	20
3.6	WHEEL ENCODERS	20
3.7	INERTIAL MEASUREMENT UNIT	21
3.8	LIDAR	21
3.9	GPS	22
3.10	BLUETOOTH	22
3.11	OAK-D-LITE CAMERA	22
3.12	POWER	23
4	ROS ARCHITECTURE	24
4.1	NETWORK AND MASTER	24
4.2	PACKAGES AND WORKSPACES	24
4.3	NODES AND TOPICS	25
4.4	COORDINATE FRAMES	26
4.5	BRINGUP SEQUENCE	27
4.6	BASE NODE	27
4.7	PS4 TELEOP	27
4.8	CHASE NODE	28
5	DRIVE KINEMATICS AND WHEEL ODOMETRY	30
5.1	NOTATION	30
5.2	FROM ENCODER COUNTS TO WHEEL TRAVEL	30
5.3	SKID-STEER KINEMATICS	31
5.4	ODOMETRY INTEGRATION	31
5.4.1	Odometry uncertainty	32
5.5	FROM VELOCITY COMMAND TO MOTOR DUTY	33
5.6	WHY SKID-STEER TURNING NEEDS SO MUCH POWER	34
6	MOTOR CONTROL AND TRACKING LOOPS	35
6.1	THE ACTUATOR CHAIN	35
6.2	DC MOTOR MODEL	36
6.3	LOOPS ON THE ROBOT TODAY	37
6.4	ENCODER SPEED ESTIMATION	38
6.5	PROPOSED: WHEEL-SPEED LOOP	38
6.6	PROPOSED: YAW-RATE LOOP	39
6.7	IMPLEMENTATION NOTES	39

7	ATTITUDE AND HEADING	41
7.1	SENSOR FRAME	41
7.2	SENSOR ERROR MODEL	41
7.3	GYROSCOPE HEADING	42
7.3.1	Bias removal at start-up	42
7.3.2	Heading error growth	42
7.3.3	Zero-velocity update	42
7.4	TILT FROM THE ACCELEROMETER	43
7.5	MAGNETOMETER COMPASS	43
7.5.1	Measurement model	43
7.5.2	Planar calibration	44
7.5.3	Heading from the field	45
7.5.4	Tilt compensation	45
7.5.5	Limits indoors	45
7.6	FUSING GYRO AND COMPASS	46
8	GPS MEASUREMENT MODEL	47
8.1	NMEA TO NAVSATFIX	47
8.2	GEODETIC TO LOCAL COORDINATES	48
8.3	THE NAVSAT TRANSFORM	48
9	SENSOR FUSION: THE GPS EKF	50
9.1	THE DISCRETE EKF	50
9.2	STATE AND PROCESS MODEL	50
9.3	MEASUREMENTS	51
9.4	OBSERVABILITY AND BEHAVIOUR WITHOUT A FIX	52
9.5	RELATION TO THE SLAM MAP	52
10	MAPPING: LIDAR SLAM	54
10.1	OCCUPANCY GRID	54
10.2	GMAPPING	54
10.2.1	Rao–Blackwellized particle filter	54
10.2.2	When scans are processed	55
10.2.3	Parameters	56
10.3	HECTOR MAPPING	56
10.4	SCAN RATE AND ROTATION	57
10.5	DIAGNOSING THE SLAM DRIFT OF 1 OCTOBER	57
11	OAK-D-LITE DEPTH CAMERA	59
11.1	HARDWARE	59
11.2	PINHOLE CAMERA MODEL	60

11.3 STEREO DEPTH	60
11.4 SPATIAL DETECTION	61
11.5 IMAGE GEOMETRY OF THE DETECTOR INPUT	62
11.6 CALIBRATION	63
11.7 MOUNTING AND FRAMES	63
12 CHASE: FOLLOWING A DOG OR PERSON	64
12.1 PIPELINE	64
12.2 OBJECT RECOGNITION: MOBILENET-SSD	65
12.3 FROM DETECTIONS TO A TRACK	66
12.4 CONTROL LAW	67
12.4.1 Bearing loop	67
12.4.2 Yaw-rate loop and gyro-stabilized bearing	69
12.4.3 Range loop	70
12.5 ARBITRATION AND SAFETY	70
12.6 BRING-UP PLAN	70
12.7 PROPOSED IMPROVEMENTS	71
13 OPERATING MODES	72
13.1 LOCALIZATION AND MAPPING MODES	72
13.2 WHAT HAPPENS WHEN A SENSOR IS MISSING	73
13.3 DRIVING MODES	73
13.4 AUTONOMOUS EXPLORATION	74
14 AUTONOMOUS MODE: SEARCH, MAP AND CHASE	76
14.1 BEHAVIOUR	76
14.2 ARCHITECTURE	76
14.3 TARGETS AND THE KID CLASS	77
14.4 SAVED MAP OR A FRESH MAP EACH TIME	78
14.5 SEARCH POINTS	78
14.6 RUNNING IT	79
15 KNOWN ISSUES AND NEXT STEPS	80
15.1 OPEN ISSUES	80
15.2 SUGGESTED IMPROVEMENTS	81
A REFERENCE	82
A.1 REPOSITORY MAP	82
A.2 BASE NODE PARAMETERS	82
A.3 SYMBOLS	82
A.4 BUILDING THIS DOCUMENT	82

List of Figures

1.1	Physical and network connections	10
2.1	Robotics Cape (SD-101D) blocks	14
4.1	ROS computation graph (gmapping mode with GPS fusion)	25
4.2	Transform tree	26
6.1	Path from a velocity command to the wheels (one side). There is no speed feedback today: the encoders feed odometry only.	35
12.1	Chase pipeline. Everything inside the dashed box runs on the camera; the BeagleBone sees only detections.	64

List of Tables

1.1	Hardware	9
1.2	Subsystem status, 2 October 2026	11
2.1	Chassis geometry (measured 2026-10-01)	12
2.2	BeagleBone AI	13
2.3	Robotics Cape connectors and what is plugged into them	15
2.4	MPU-9250 characteristics used here (datasheet rev. 1.1)	16
2.5	5 V rail load (estimates from datasheets)	17
3.1	Device tree changes on top of upstream	19
3.2	UART assignments (confirmed on the board)	19
3.3	Encoder and motor channel mapping (spin and lifted-wheel tests, 2026-10-01)	21
4.1	Catkin packages	24
4.2	Topics (message types without their package prefix)	25
4.3	PS4 controller mapping (<code>hid-sony</code> , kernel 4.14)	28
5.1	Geometry and drive parameters	30
5.2	Commanded duty (D_L, D_R) for typical commands	33
6.1	Periodic, feedback and tracking loops	37
9.1	EKF inputs (<code>gps_ekf.yaml</code>)	51
10.1	gmapping parameters (<code>laptop/bbai_slam/gmapping_bbai.launch</code>)	56
11.1	OAK-D-Lite specifications (Luxonis datasheet)	59
11.2	Stereo depth resolution, $fB = 33.4$ px m, $\delta d = 1$ px	61
11.3	Calibrated intrinsics (from the flashed calibration file)	63
12.1	Chase states	67
12.2	Chase parameters (<code>chase.launch</code>)	68
12.3	Bearing-loop phase margin, $\tau_e = 0.14$ s	68

13.1	Localization and mapping modes	72
13.2	Degradation by sensor	73
14.1	Autonomous-mode nodes.	77
A.1	Where things are in <code>jstumfoll2/bbai-robot</code>	82
A.2	<code>base.yaml</code>	83
A.3	Main symbols	83

Chapter 1

SYSTEM OVERVIEW

1.1 PURPOSE AND SCOPE

This document records how the BeagleBone AI robot is built and how its software fits together. Chapter 2 describes every piece of hardware: the chassis, the BeagleBone AI, the Robotics Cape, each sensor, and the power budget. Chapter 3 records every change that was needed to bring each device up on the board, and Chapter 4 shows how the devices, computers and ROS nodes are connected. The navigation chapters derive the wheel odometry (Chapter 5), the motor control and tracking loops (Chapter 6), the attitude and heading estimate from the gyroscope and magnetometer (Chapter 7), the GPS measurement model (Chapter 8), the extended Kalman filter that fuses them (Chapter 9), and the SLAM algorithms that build the map (Chapter 10). Chapter 11 covers the OAK-D-Lite stereo depth camera and Chapter 12 the chase behaviour that uses it to follow a dog or a person. Chapter 13 lists the operating modes that are available with or without each sensor, and Chapter 15 collects known problems, including the analysis of the SLAM drift observed on 1 October 2026.

The notation and the sensor-error models are taken from the author’s master’s thesis on the two-wheeled inverted pendulum (TWIP) [1], in particular its treatment of IMU measurements (thesis §4.3.1), its sensor noise model with a constant and a walking bias (thesis §7.5), and its discrete Kalman filter (thesis §7.7). Where this document reuses a result from the thesis it says so, so that the derivations can be read side by side.

1.2 GOALS

The goals, in Jason’s priority order as of 1 October 2026, are:

1. Back up and version all of the board’s work (done: this repository and the `librobotcontrol` fork).
2. Chase the dog using on-camera detection on the OAK-D-Lite.
3. Map the house with lidar SLAM.

4. Drive manually with a PS4 controller over Bluetooth.
5. Publish the IMU, wheel odometry and GPS as ROS topics and fuse them into a single navigation state.

1.3 PLATFORM SUMMARY

The robot is a small four-wheel skid-steer chassis. All four wheels are driven, each by its own DC gear motor with a Hall-effect encoder. A BeagleBone AI with a Robotics Cape is the only on-board computer. Table 1.1 lists the hardware and Figure 1.1 shows how it is connected.

Table 1.1: Hardware

Item	Details
Computer	BeagleBone AI (TI AM5729: 2× Cortex-A15 at 1.5 GHz, 4× PRU, 2× C66x DSP, 4× EVE). Debian 10 Buster, kernel 4.14.108-ti-r136, Python 3.7, ROS Melodic built from source.
Cape	Robotics Cape (Beaglebone Blue functionality as a cape): 4 H-bridge motor channels, 4 encoder inputs, MPU-9250 IMU, barometer, UART and I ² C headers, 2-cell LiPo input and charger.
Drive	4-wheel skid steer. Wheel diameter 79.2 mm, left-right wheel centres 120 mm, axle spacing 101 mm.
Encoders	DAGU 8-pole magnet rings read by Allegro A1230 dual Hall sensors (open-drain outputs). Measured 15 counts per wheel revolution.
IMU	InvenSense MPU-9250 on the cape: 3-axis gyroscope, accelerometer and AK8963 magnetometer.
Lidar	SLAMTEC RPLIDAR A1M8 on a cape UART (/dev/ttyS3), about 7.2 Hz scan rate, 0.15 m to 12 m range.
GPS	GlobalSat EM-506 (SiRF Star IV) on /dev/ttyS2, NMEA at 4800 baud, 1 Hz.
Camera	Luxonis OAK-D-Lite (stereo depth plus RGB, Myriad X VPU) on the BeagleBone's USB-A port.
Radio	On-board Wi-Fi and Bluetooth (AzureWave AW-CM256SM, Broadcom BCM43455); Bluetooth HCI on UART6 (/dev/ttyS5).
Controller	Sony DualShock 4 (PS4) over Bluetooth.
Power	2-cell LiPo pack (8.17 V full) into the cape, or a bench supply.
Off-board	Ubuntu 20.04 laptop with ROS Noetic (SLAM, sensor fusion, RViz); Windows PC (development, camera calibration, board serial console on COM3).

1.4 DIVISION OF WORK BETWEEN THE COMPUTERS

The board runs everything that touches hardware: the ROS master, the lidar driver, the motor, encoder and IMU node, the GPS node, the PS4 teleop node and the chase node. The OAK-D-Lite runs its neural network on its own Myriad X, so the board only receives detections. The laptop runs everything that is heavy but not time-critical: SLAM, the EKF and navsat transform, and RViz. The laptop joins the board's ROS master over Wi-Fi (ROS_MASTER_URI=http://192.168.3.120:11311).

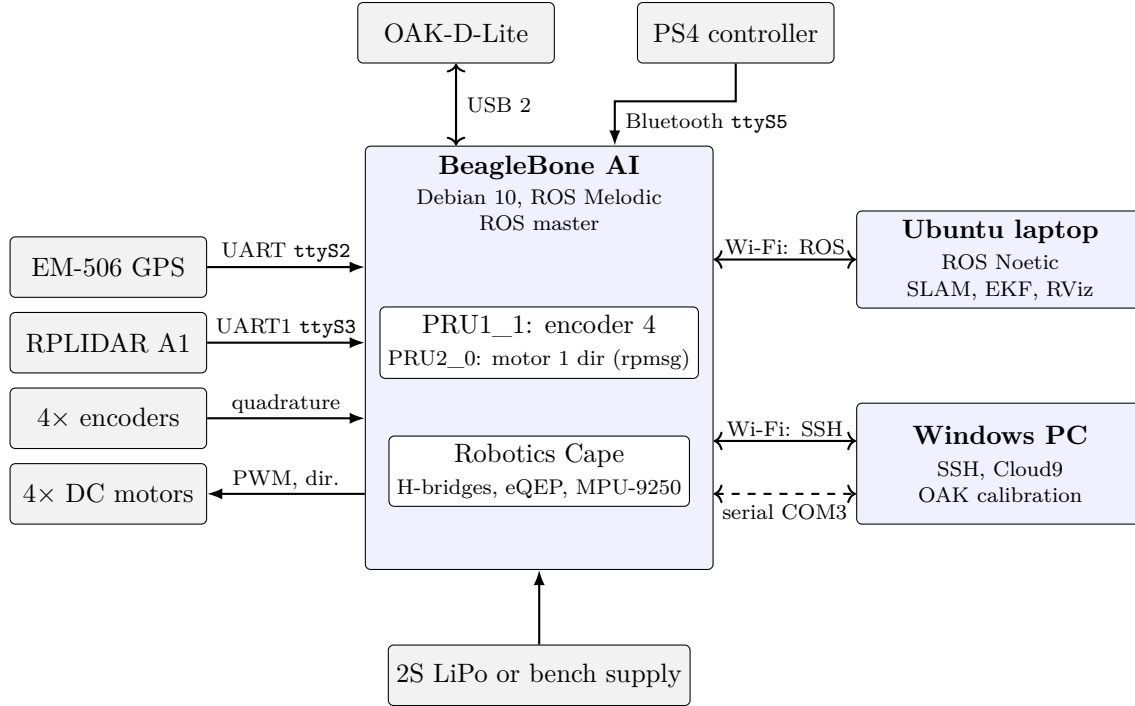


Figure 1.1: Physical and network connections

This split follows a decision made on 1 October 2026: keep everything on the BeagleBone where possible, run SLAM on the laptop, and treat a Raspberry Pi 5 as the fallback compute if the board runs out of headroom.

1.5 STATUS

In flux. SLAM tuning, the EKF configuration and the teleop turn curve were all being changed on 2 October 2026. Each is marked where it appears.

Table 1.2: Subsystem status, 2 October 2026

Subsystem	State	Notes
Motors (all four)	Working	Motor 1 needs the PRU2_0 firmware (§3.5).
Encoders (all four)	Working	Needed pull-ups in the device tree (§3.2).
Wheel odometry	Working	0.914 m measured for a 0.914 m push.
IMU (gyro, accel)	Working	Heading drift was 1.6 °/min parked; zero with the new ZUPT.
Magnetometer	Calibrated	Compass agrees with the gyro to 3% over two turns.
GPS	Working	No fix indoors, as expected.
Lidar	Working	7.2 Hz on <code>ttys3</code> .
Bluetooth + PS4	Working	Report stalls of 100–500 ms in bursts.
SLAM (gmapping)	Improving	Parked drift fixed; lidar found mounted backward and its transform corrected (§10.5).
GPS fusion (EKF)	Built, untested	Needs an outdoor run with a fix.
Autonomous exploration	Built, untested	Frontier exploration with <code>move_base</code> (§13.4).
OAK-D-Lite detection	Working	Resets the board under full load; waiting on separate camera power.
Chase controller	Dry run only	Waiting on camera power.

Chapter 2

HARDWARE REFERENCE

This chapter describes every piece of hardware on the robot: what it is, its relevant specifications, and how it is connected. The next chapter covers the software changes that were needed to make each piece work on the BeagleBone AI.

2.1 CHASSIS AND DRIVETRAIN

The chassis is a small four-wheel-drive platform bought from SparkFun, with an aluminium plate added as a deck for the electronics. Each wheel is driven by its own brushed DC gear motor, and the two motors on each side are commanded together, which makes the robot a skid-steer vehicle (§5.3). Table 2.1 gives the measured geometry.

Table 2.1: Chassis geometry (measured 2026-10-01)

Quantity	Value	Note
Wheel diameter	79.2 mm	radius $r = 39.6$ mm
Track (left–right wheel centres)	120 mm	b
Wheelbase (front–back axles)	101 mm	L ; $L/b = 0.84$
Encoder resolution	15 counts per wheel turn	16.6 mm of travel per count
Top speed (wheels lifted, duty 1)	about 1.2 m/s	capped to about 0.65 m/s in software

The encoders are DAGU magnetic encoder kits: an 8-pole ring magnet on each wheel hub read by an Allegro A1230 dual Hall sensor. Because the magnet turns with the wheel, the encoder measures the wheel itself, after the gearbox. This is the opposite trade from the TWIP, whose encoders were on the motor shaft ahead of the gearbox (thesis §4.2.3): here gear backlash is seen by the encoder, but the resolution is about 100 times coarser per wheel turn.

2.2 BEAGLEBONE AI

The BeagleBone AI is built around the Texas Instruments AM5729 system on chip (the Sitara AM57x family). It keeps the BeagleBone Black’s form factor and its two 46-pin expansion headers,

P8 and P9, so BeagleBone capes physically fit. Table 2.2 summarises the parts that matter for the robot.

Table 2.2: BeagleBone AI

Block	Details and use on the robot
CPU	2× ARM Cortex-A15 at 1.5 GHz. Runs Linux, ROS and all the Python nodes.
DSP	2× TI C66x floating-point DSP. Unused.
EVE	4× Embedded Vision Engine vector processors, used by TI’s TIDL deep-learning library. Used for the 2022 webcam classification demo; the OAK-D-Lite has since taken over vision.
PRU-ICSS	2 industrial communication subsystems, each with 2 PRU cores at 200 MHz (4 in total). PRU1_1 counts encoder 4; PRU2_0 drives motor 1’s direction pin (§3.5).
IPU	2× dual Cortex-M4. Unused.
Memory	1 GB DDR3, 16 GB eMMC (boots Debian), microSD slot.
Peripherals	eQEP quadrature decoders (3 used for encoders 1–3), ePWM (motor PWM), UARTs, I ² C, SPI, GPIO, ADC.
Connectivity	Gigabit Ethernet; AzureWave AW-CM256SM module (Broadcom BCM43455): dual-band Wi-Fi on SDIO and Bluetooth on UART6; USB Type-C (power and USB 3 device/host); USB Type-A host (USB 2, used by the OAK-D-Lite).
Power	5 V in through USB-C, or through the headers’ SYS_5V pins, which is how the Robotics Cape powers it. A TPS659037 PMIC makes the internal rails.
Cooling	A fan (from the parts list) on the heat sink; the AM5729 runs hot under load.

Header multiplexing. On the BeagleBone Black each header pin connects to one ball of the AM335x. On the BeagleBone AI many header pins are wired to *two* AM5729 balls (written P9.19a and P9.19b, for example), to reach more functions. Only one of the two may be driven; the other must be set to an input or to mode 15 (off) in the device tree, or the two outputs fight. Each ball has up to 16 multiplexer modes; the robot uses mode 10 for eQEP and ePWM, mode 12 and 13 for PRU inputs and outputs, mode 14 for GPIO and modes 1–7 for UART and I²C. The PRU pin map that guided the motor 1 fix is in `claudefiles/beagleboneAI/PRUs/` in the project files: it shows that P9.13’s reachable ball (C17) offers `pr2_pru0_gpo15` in mode 13 and no ordinary GPIO.

2.3 ROBOTICS CAPE

The Robotics Cape (Strawson Design SD-101D, revision D, sold as BB-CAPE-ROBOTICS) was designed for the BeagleBone Black. It carries the robot’s power system, motor drivers, IMU and connectors. Figure 2.1 shows its blocks, and the schematic is in `claudefiles/RoboticsCape/` in the project files.

2.3.1 Power

Inputs. A 2-cell LiPo on a 3-pin JST-XH connector (V_BATT plus the cell midpoint for balancing), and a 5.5 mm×2.1 mm centre-positive barrel jack for 9 V to 18 V DC (VDC). A 9 V, 2 A adapter

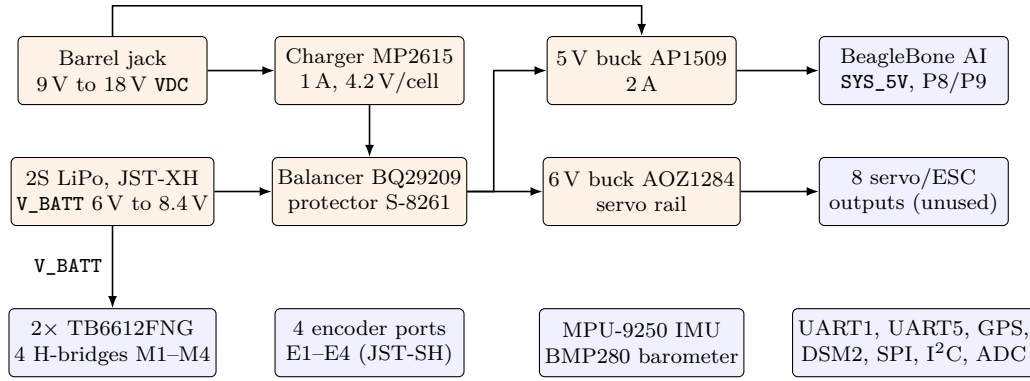


Figure 2.1: Robotics Cape (SD-101D) blocks

is on the parts list. Either input can run the robot.

Charger. An MPS MP2615 charges the pack from VDC at 1 A (set by a $0.1\ \Omega$ sense resistor) to 4.2 V per cell.

Protection. A TI BQ29209 balances the two cells and a Seiko S-8261 cuts the pack off at 4.3 V per cell (overcharge) or 2.5 V per cell (over-discharge).

5 V rail. A Diodes AP1509 buck regulator, fed from VDC or V_BATT through diodes, makes 5 V at up to 2 A. This rail powers the BeagleBone through SYS_5V, and through it the BeagleBone’s USB ports and everything plugged into them. Its 2 A limit is central to the board resets (§2.5).

6 V rail. An AOZ1284 buck powers the servo headers when enabled by SERVO_EN. Unused.

Monitoring. Two dividers of $47\ \text{k}\Omega$ and $4.7\ \text{k}\Omega$ scale VDC and V_BATT by 1/11 into the 1.8 V ADC. Four LEDs show battery level.

2.3.2 Motor drivers

Two Toshiba TB6612FNG dual H-bridges drive the four motor ports (H1: M1 and M2; H2: M3 and M4) directly from V_BATT. Each channel has two direction inputs (IN1, IN2, the cape’s MDIRxA/B) and a PWM input, and a shared standby line (MOT_STBY) enables both chips. Per channel the TB6612FNG is rated 1.2 A continuous and 3.2 A peak, at up to 15 V. Its truth table is

IN1	IN2	PWM	Output
H	L	H	forward (current one way)
L	H	H	reverse
H	H	x	short brake
L	L	H	coast (outputs off)
x	x	L	short brake

so speed is set by PWM duty with the direction held on IN1/IN2 (sign–magnitude drive). During the PWM off-time the bridge short-brakes the motor, which makes average speed close to proportional to duty. librobotcontrol runs the PWM at 25 kHz.

2.3.3 Encoder ports

Four 4-pin JST-SH connectors, E1–E4, each carry GND, 3.3 V and the A and B signals. E1–E3 go to three eQEP hardware decoders; E4 goes to PRU inputs (on the BeagleBone Black it was read by PRU firmware too, since the AM335x has only three eQEPs). There are no pull-up resistors on these lines on the cape, which is why the device tree must enable the SoC’s internal ones (§3.2).

2.3.4 Sensors on the cape

The MPU-9250 IMU sits on the cape’s I²C2 bus with its interrupt on IMU_INT (P9.25). A Bosch BMP280 barometer shares the bus at address 0x76; it is not used. The I²C buses have 4.7 k Ω pull-ups.

2.3.5 Connectors

Table 2.3: Robotics Cape connectors and what is plugged into them

Connector	Type	Pins	On this robot
M1–M4	JST-ZH 2-pin	motor $\pm$	M1 left back, M2 left front, M3 right front, M4 right back
E1–E4	JST-SH 4-pin	GND, 3.3V, A, B	encoders: ch1 left back, ch2 right back, ch3 left front, ch4 right front
UART1	JST-SH 4-pin	GND, 3.3V, RX, TX	RPLIDAR A1 data (/dev/ttyS3)
GPS	JST-SH 6-pin	GND, 3.3V, RX, TX, 5V, GND	EM-506 (/dev/ttyS2)
UART5	JST-SH 4-pin	GND, 3.3V, RX, TX	free
DSM2	JST-ZH 3-pin	RX, GND, 3.3V	free (radio receiver was considered)
I ² C1, SPI1.1/1.2, ADC	JST-SH		free
PWR	JST-SH 4-pin	5V, 3.3V, GND	free; auxiliary power
Servo 1–8	0.1” 3-pin	signal, 6V, GND	free
LiPo / jack	JST-XH 3-pin / barrel		battery / bench supply

The cape’s buttons (Pause, Mode) and LEDs (red, green, battery) are routed to P8 GPIOs; the buttons do not work on the BeagleBone AI yet.

Table 2.4: MPU-9250 characteristics used here (datasheet rev. 1.1)

	Value	Note
Gyro range	$\pm 250^\circ/\text{s}$ (131 LSB per $^\circ/\text{s}$)	default in librobotcontrol
Gyro rate-noise density	$0.01^\circ/\text{s}/\sqrt{\text{Hz}}$	
Gyro zero-rate offset	$\pm 5^\circ/\text{s}$ initial, $\pm 30^\circ/\text{s}$ over temperature	hence start-up bias removal
Accel noise density	$300\mu\text{g}/\sqrt{\text{Hz}}$	
Magnetometer (AK8963)	$\pm 4800\mu\text{T}$ range	axes differ from gyro; library realigns

2.4 SENSORS

2.4.1 MPU-9250 IMU

2.4.2 RPLIDAR A1M8

A triangulation laser scanner on a rotating head driven by its own motor. It measures up to 8000 times per second, 0.15 m to 12 m, with resolution under 1% of distance, over 360 degrees. Scan rate is 1–10 Hz (5.5 Hz typical); on this robot it settles at about 7.2 Hz. The scanner core needs 4.9 V to 5.5 V with low ripple (about 300 mA running, 500 mA at start-up) and the motor 5 V to 9 V. Data is 115 200 baud serial at 3.3 V, which goes to the cape’s UART1 header; the cape’s UART headers carry only 3.3 V, so the lidar’s power comes from a separate 5 V feed.

2.4.3 EM-506 GPS

A GlobalSat module with a SiRF Star IV chipset and built-in patch antenna: -163 dBm tracking sensitivity, under 2.5 m horizontal error (50%, open sky), cold start under 35 s (hot under 1 s), NMEA 0183 at 4800 baud, 1 Hz, SBAS (WAAS) capable. It draws about 50 mA from 4.5 V to 6.5 V, which the cape’s GPS header provides.

2.4.4 Wheel encoders (A1230)

Each A1230 contains two Hall switches 1 mm apart whose outputs are in quadrature when they face a matching ring magnet. Outputs are open-drain, sinking up to 20 mA, and need an external pull-up; supply is 3.3 V to 18 V. Decoding the four edges of the two channels gives $4\times$ the number of pole pairs per revolution.

2.4.5 OAK-D-Lite camera

A stereo depth and AI camera; Chapter 11 covers it in detail.

2.4.6 PS4 controller

A Sony DualShock 4 over Bluetooth Classic HID, read through the kernel's `hid-sony` driver as `/dev/input/js0`.

2.5 POWER BUDGET

Everything except the motors runs from the cape's 5 V rail, which is limited to 2 A by the AP1509. Table 2.5 estimates the load.

Table 2.5: 5 V rail load (estimates from datasheets)

Load	Current at 5 V	Source
BeagleBone AI, both A15 cores busy, Wi-Fi and BT	0.8 A to 1.2 A	typical for the board
OAK-D-Lite running depth and detection	0.8 A to 1.2 A (1.5 max)	datasheet: 4 W to 6 W
RPLIDAR A1 core + motor	0.4 A to 0.6 A	datasheet
GPS, IMU, encoders	0.07 A	datasheets
Total	2.1 A to 3.1 A	vs. 2 A regulator

The estimated total exceeds the regulator's rating once the camera runs alongside everything else, which matches what was observed: the camera alone, or everything without the camera, ran fine, while the full load reset the board with the battery voltage steady. The camera datasheet anticipates this: hosts that cannot supply its current should use a USB Y-adaptor that feeds the camera from a separate supply. That is the planned fix (a USB splitter cable and battery bank), which moves 0.8 A to 1.5 A off the cape's rail.

The motors draw from `V_BATT` directly through the H-bridges, not from the 5 V rail. Their start-up and stall currents can still sag the battery and the regulator's input, which is another reason to keep the 5 V load well below 2 A.

Chapter 3

HARDWARE BRING-UP

This chapter records every change that was needed to make each device work on the BeagleBone AI. The Robotics Cape was designed for the BeagleBone Black and Blue, whose AM335x processor has a different pin multiplexer, different GPIO numbering and different peripheral addresses from the AM5729 on the BeagleBone AI. Almost every problem in this chapter comes from that mismatch.

3.1 OPERATING SYSTEM AND BOOT

The board runs the 2020-04-06 TI image (Debian 10 Buster, kernel 4.14.108-ti-r136). Debian 10 is out of support, so `/etc/apt/sources.list` points at archive.debian.org with the security line commented out; without that, `apt` cannot install anything (it was needed for `libatlas-base-dev`, which `depthai` uses).

The boot configuration is `system/uEnv.txt`, installed as `/boot/uEnv.txt`:

```
uname_r=4.14.108-ti-r136
dtb=am5729-beagleboneai-roboticscape.dtb
cmdline=coherent_pool=1M
```

The `dtb=` line makes U-Boot load the custom Robotics Cape device tree instead of the stock one. Journald was made persistent (`/var/log/journal`) on 1 October 2026 so that the cause of an unexpected reboot survives the reboot.

3.2 DEVICE TREE

The device tree is built from `beagleboard/BeagleBoard-DeviceTrees` (branch `v4.14.x-ti`, commit `4a9c0a6`) plus `devicetree/roboticscape-bbai.patch`. The full source files are also kept in `devicetree/`. Table 3.1 lists every change and why it was made.

The patch also moves the `epwmss2/ehrpwm2` and UART nodes around without changing what they enable, and adds a model string.

Table 3.1: Device tree changes on top of upstream

Change	Reason
<code>&hdm</code> disabled	HDMI shares pins with cape signals.
<code>&i2c4</code> enabled on P9.19/P9.20 (mode 7), shared balls set to GPIO input	Cape I ² C2 header. The BB-AI routes two SoC balls to each of these header pins, so the unused ball must be parked.
P9.13 muxed to <code>pr2_pru0_gpo15</code> (mode 13)	Motor 1 direction B. The AM335x GPIO this pin used is not available on the AM5729 ball that reaches the header; the only usable function is a PRU output (§3.5).
<code>PIN_INPUT_PULLUP</code> on all 8 encoder pins (eQEP1A/B, eQEP2A/B, eQEP3A/B, PRU1 gpi16/18)	The A1230 outputs are open-drain and need a pull-up. Without it no encoder counted at all (2026-10-01).
<code>&uart6</code> enabled	Bluetooth HCI for the BCM43455. It had been commented out in the base <code>.dts</code> in 2022 for an unrecorded reason; re-enabled and tested on 2026-10-01.

Build and install. On the board (needs `sudo` and a reboot):

```
cd ~/src/BeagleBoard-DeviceTrees && git apply .../roboticscape-bbai.patch
make src/arm/am5729-beagleboneai-roboticscape.dtb
sudo cp /boot/dtbs/4.14.108-ti-r136/am5729-beagleboneai-roboticscape.dtb ~/roboticscape.dtb.bak
sudo cp src/arm/am5729-beagleboneai-roboticscape.dtb /boot/dtbs/4.14.108-ti-r136/
sudo reboot
```

The backup in the home folder is the rollback path; the board’s eMMC can also be repaired from a microSD boot if the new tree does not boot.

3.3 SERIAL PORTS

The cape’s UART header names, the AM5729 UART instances and the Linux device names do not line up, which caused the lidar to be looked for on the wrong port at first. Table 3.2 gives the confirmed mapping.

Table 3.2: UART assignments (confirmed on the board)

Device	Cape header	Linux device	Settings
RPLIDAR A1	UART1	<code>/dev/ttyS3</code> (old name <code>ttty03</code>)	115 200 baud
EM-506 GPS	GPS	<code>/dev/ttyS2</code> (SoC UART3)	4800 baud 8N1
Bluetooth HCI	(on board)	<code>/dev/ttyS5</code> (SoC UART6)	3 Mbaud, RTS/CTS
Debug console	(on board)	<code>tttyS0</code>	115 200 baud, PC COM3

3.4 LIBROBOTCONTROL FORK

Motors, encoders and the IMU are driven through `librobotcontrol` (used from Python via `rcpy`). The upstream library supports the BeagleBone AI only partially. The fork `jstum-foll2/librobotcontrol`, branch `v1.1_AIfixes`, is a submodule of this repository and carries two commits on top of upstream `99a08db`: Jason’s 2022 work (`d90e24b`) and the uncommitted board changes recovered on 2026-10-01 (`593182b`). The functional changes are:

- **GPIO numbering** (`motor.c`). All motor direction pins and the motor standby pin were remapped to AM5729 GPIO banks (for example MDIR1A is `gpio5_0`, MOT_STBY is `gpio6_20`). The per-channel polarity table was changed to $\{+1, -1, +1, -1\}$.
- **A virtual GPIO chip for the PRU** (`gpio.c`). Chip number 11 is redirected to the character device `/dev/rpmsg_pru32`. Setting a pin on chip 11 writes the ASCII character ‘0’ or ‘1’ to that device instead of issuing a GPIO ioctl. Motor 1’s direction B pin is defined as chip 11, so `rc_motor_init()` fails unless the PRU firmware below is running.
- **eQEP sysfs paths** (`encoder_eqep.c`). Updated to the AM5729 addresses (`4843e000.epwmss` and so on).
- **IMU bus** (`mpu.c`). The MPU-9250 is on I²C bus 3 with its interrupt on `gpio5_17` (P9.25).
- **PRU encoder** (`encoder_pru.c`). Encoder channel 4 is counted by firmware on PRU1_1 (`am57xx-pru1_1-rc-encoder-fw-c`), read from PRU shared memory at offset 0.

3.5 MOTOR 1 DIRECTION FIRMWARE ON PRU2_0

The PRU firmware `gpio_rpmsg.pru2_0.c` (`librobotcontrol/pru_firmware/customfw/`) is about 100 lines. It opens an RPMsg channel named `rpmsg-pru` on port 32, which the kernel exposes as `/dev/rpmsg_pru32`. Each message received is parsed with `atoi`; a non-zero value sets bit 15 of the PRU output register R30 (P9.13), and zero clears it. After each message the firmware waits $10\,200 + 4\,000\,000$ PRU cycles, about 20 ms at 200 MHz, so motor 1 can reverse direction at most about 50 times per second. That is far faster than any command the robot sends.

The firmware is installed as `/lib/firmware/am57xx-pru2_0-fw`. It is not started at boot; `base_node.py` starts it before initialising the motors, equivalent to

```
echo am57xx-pru2_0-fw > /sys/class/remoteproc/remoteproc2/firmware
echo start > /sys/class/remoteproc/remoteproc2/state
```

and then waits up to 5 s for `/dev/rpmsg_pru32` to appear.

3.6 WHEEL ENCODERS

Each wheel has a DAGU 8-pole magnet ring (4 north, 4 south) read by an Allegro A1230 dual-channel Hall sensor in quadrature. Two facts from the datasheets explained why nothing counted at first:

- The A1230 outputs are open-drain, and the datasheet requires pull-up resistors. The cape has none on these lines, so the internal pull-ups were enabled in the device tree (Table 3.1).
- The A1230 needs at least 3.3 V; the cape’s encoder headers supply exactly 3.3 V, which is in specification but at the bottom of the range.

An 8-pole ring with a quadrature sensor gives $4 \times 4 = 16$ edges per revolution in principle. A 3 ft (0.914 m) push measured about 55 counts on every wheel, all four within 2 counts of each other, giving 15.0 counts per revolution, and that value is in `base.yaml`.

Table 3.3: Encoder and motor channel mapping (spin and lifted-wheel tests, 2026-10-01)

Wheel	Encoder channel	Counter	Encoder sign	Motor channel
Left back	1	eQEP	+1	M1
Right back	2	eQEP	−1	M4
Left front	3	eQEP	+1	M2
Right front	4	PRU1_1	−1	M3

All four motors drive forward with positive duty (`motor_signs` all +1), after the library’s own polarity table. The right-side encoders count down when driving forward because they are mounted mirrored, which the encoder signs correct.

3.7 INERTIAL MEASUREMENT UNIT

The MPU-9250 is read through `rcpy.mpu9250` with the magnetometer enabled. `rcpy` returns acceleration in m/s^2 , angular rate in $^\circ/\text{s}$ and magnetic field in μT ; `base_node.py` converts to SI (rad/s, T) before publishing. Relevant datasheet values at the default $\pm 250^\circ/\text{s}$ range are a rate-noise spectral density of $0.01^\circ/\text{s}/\sqrt{\text{Hz}}$, an initial zero-rate offset of up to $\pm 5^\circ/\text{s}$, and a zero-rate offset that can move by up to $\pm 30^\circ/\text{s}$ over the full temperature range. The last figure is why the gyroscope bias must be measured at every start (§7.3).

The magnetometer read about $150\mu\text{T}$ on the robot, against Earth’s roughly $50\mu\text{T}$. The extra field comes from the motor magnets, the encoder magnet rings and current in the wiring, which is why it needs the calibration in §7.5.

3.8 LIDAR

The RPLIDAR A1M8 is wired to the cape’s UART1 header and appears as `/dev/ttyS3`. It reports healthy and publishes `/scan` at about 7.2 Hz through `rplidar_ros` (catkin workspace `~/melodic-catkin-ws`). The A1 scans clockwise; the driver converts to the counter-clockwise ROS convention.

3.9 GPS

The EM-506 is a SiRF Star IV receiver that outputs NMEA 0183 at 4800 baud on `/dev/ttyS2`. Two pieces of software read it:

- `gps/libgps-bbai-ttyS2-4800.patch`: a patch to the small C library `wdalmut/libgps` that changes its port and baud rate. This was the 2022 test path.
- `ros/bbai_base/scripts/gps_node.py`: the ROS publisher used now (§8.1). It configures the UART itself with `termios`.

Indoors the receiver reports no fix; a fix needs open sky.

3.10 BLUETOOTH

The on-board BCM43455 Bluetooth controller talks HCI over SoC UART6. Three things were needed to bring it up:

1. `&uart6` enabled in the device tree (Table 3.1). It then appears as `/dev/ttyS5`.
2. `system/blacklist-btsdio.conf` in `/etc/modprobe.d/`. The `btsdio` driver otherwise creates a non-working `hci0` on the SDIO bus, so the real controller would become `hci1`.
3. `ros/bbai_teleop/bt-up.sh`, run at boot by `bbai-bluetooth.service`. It finds the `tty` whose device is `48068000.serial`, attaches it with `hciattach <tty> bcm43xx 3000000 flow` (retrying at 3Mbaud if the chip was already initialised), brings `hci0` up, disables sniff mode on new links and turns off Wi-Fi power saving. The last two keep the controller’s input reports at full rate, because the BCM43455 shares one radio between Wi-Fi and Bluetooth.

The PS4 controller is paired once with `pair-ps4.sh`, which runs the whole `bluetoothctl` conversation (agent, scan, pair, trust, connect) in a single session. The kernel’s `hid-sony` driver then creates `/dev/input/js0`.

3.11 OAK-D-LITE CAMERA

The camera runs with `depthai` 2.19.1 on the board’s Python 3.7 over USB 2, at about 15fps for RGB plus depth. `MobileNet-SSD` runs on the camera’s Myriad X and detects the dog at 93–99% confidence, about 10 detections per second. The camera’s EEPROM calibration was found blank; it was self-calibrated on the PC with `depthai v2` tools and flashed on 2026-10-01 (epipolar error 0.36, baseline 7.43 cm). The procedure is in `docs/CALIBRATION.md` and the backup is in `camera/pc-tools/`. Chapter 11 covers the camera’s geometry, depth and calibration in detail.

3.12 POWER

The board reset several times on 1 October 2026. Persistent journald, a once-per-second battery logger (`~/power.log`) and the serial console narrowed the cause down:

- The battery input held steady (7.7 V to 8.2 V) through resets.
- With the motor driver and lidar stopped, the camera ran detection for 90 s with no reset.
- The console showed the camera's USB disconnecting and reconnecting immediately before a reset, with no kernel panic.

The conclusion is that the cape's 5 V rail, which feeds the BeagleBone and its USB port, drops out when the camera (1 A to 1.5 A), motors, lidar and Bluetooth all run together. The fix in progress is to power the camera from a separate battery bank through a USB splitter cable. The load estimate behind this is in §2.5.

Chapter 4

ROS ARCHITECTURE

4.1 NETWORK AND MASTER

ROS 1 is used throughout: Melodic on the board (built from source into `/opt/ros/melodic`) and Noetic on the laptop. The two are compatible on the wire for every message type used here. The board runs the ROS master, so the board must be brought up first. Both machines set

```
ROS_MASTER_URI=http://192.168.3.120:11311
ROS_IP=<own address> # 192.168.3.120 on the board, 192.168.3.147 on the laptop
```

ROS_IP matters because ROS nodes advertise their own address to the master; without it the laptop would advertise a hostname the board cannot resolve. On the laptop the variables are set by `laptop/bbai_slam/env.sh`.

4.2 PACKAGES AND WORKSPACES

Table 4.1: Catkin packages

Package	Workspace	Contents
bbai_base	~/melodic-catkin-ws (board)	base_node.py (motors, encoders, IMU), gps_node.py, mag_calibrate.py, motor_map_ test.py, robot.launch, base.yaml
rplidar_ros	~/melodic-catkin-ws (board)	SLAMTEC lidar driver
bbai_teleop	~/teleop_ws (board)	ps4_teleop.py, teleop.launch, Bluetooth scripts
bbai_chase	~/chase_ws (board)	chase_node.py, chase.launch
bbai_slam (scripts)	~/bbai_slam (laptop)	gmapping, hector and lidar-only launch files, GPS fusion launch, RViz config, start and reset scripts

Python 3 workspaces on the board are built with

```
catkin_make -DPYTHON_EXECUTABLE=/usr/bin/python3
```

4.3 NODES AND TOPICS

Figure 4.1 shows the computation graph and Table 4.2 lists every topic.

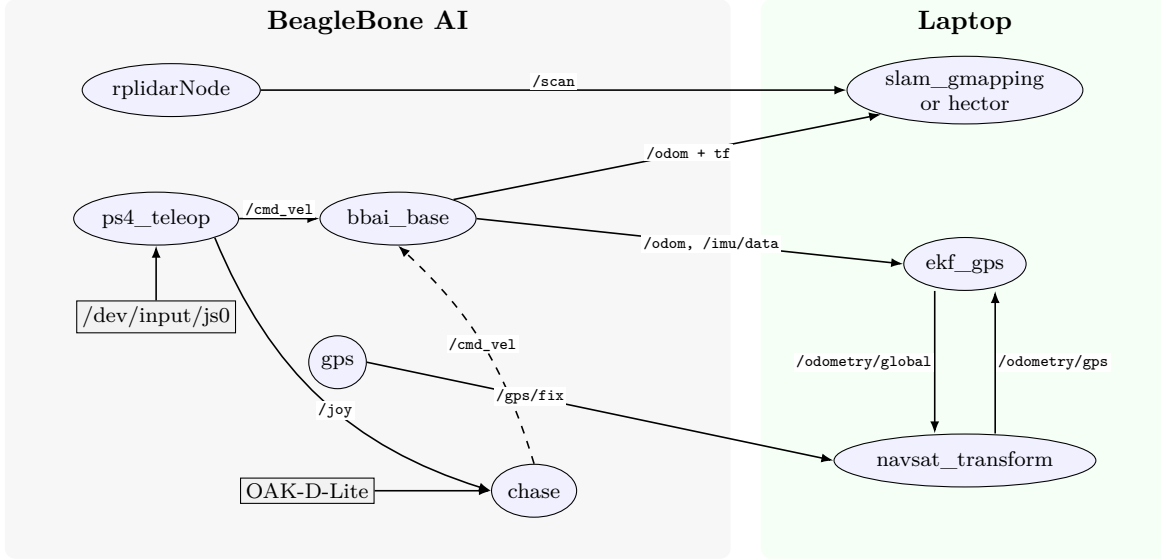


Figure 4.1: ROS computation graph (gmapping mode with GPS fusion)

Table 4.2: Topics (message types without their package prefix)

Topic	Type	Rate	Publisher → subscribers
<code>/scan</code>	LaserScan	7.2 Hz	rplidarNode → SLAM, RViz
<code>/odom</code>	Odometry	30 Hz	bbai_base → EKF, RViz
<code>/imu/data_raw</code>	Imu	30 Hz	bbai_base (rates and acceleration, no orientation)
<code>/imu/data</code>	Imu	30 Hz	bbai_base (adds compass yaw) → EKF, navsat_transform
<code>/imu/mag</code>	MagneticField	30 Hz	bbai_base (raw field, tesla)
<code>/wheel_ticks</code>	Int32MultiArray	30 Hz	bbai_base (raw counts of channels 1–4, debugging)
<code>/cmd_vel</code>	Twist	20–100 Hz	ps4_teleop, chase → bbai_base
<code>/joy</code>	Joy	20–100 Hz	ps4_teleop → chase
<code>/gps/fix</code>	NavSatFix	1 Hz	gps → navsat_transform
<code>/gps/vel</code>	TwistStamped	1 Hz	gps (ENU ground velocity, only with a fix)
<code>/odometry/global</code>	Odometry	20 Hz	ekf_gps (frame <code>gps_map</code>) → navsat_transform, user
<code>/odometry/gps</code>	Odometry	1 Hz	navsat_transform → ekf_gps

Topic	Type	Rate	Publisher → subscribers
/gps/filtered	NavSatFix	10 Hz	navsat_transform (fused estimate as latitude/longitude)
/map	OccupancyGrid	0.5 Hz	SLAM → RViz, map_saver
/chase/target	PointStamped	15 Hz	chase (target in the camera frame)
/chase/status	String	2 Hz	chase (human-readable state)
/chase/cmd_vel_ preview	Twist	15 Hz	chase (what it would command)

4.4 COORDINATE FRAMES

Frames follow ROS REP 103 and REP 105 [2, 3]: right-handed, x forward, y left, z up on the body, and east-north-up (ENU) for world frames, with yaw ψ measured counter-clockwise from the world x axis. REP 105 defines three world-fixed frames with different guarantees:

odom continuous (no jumps) but drifts without bound. Published by `bbai_base` from wheel odometry and the gyroscope.

map does not drift, but may jump when a correction arrives. Published by SLAM as the transform `map→odom`.

earth/UTM the GPS frame, related to `map` by the navsat transform (§8.3).

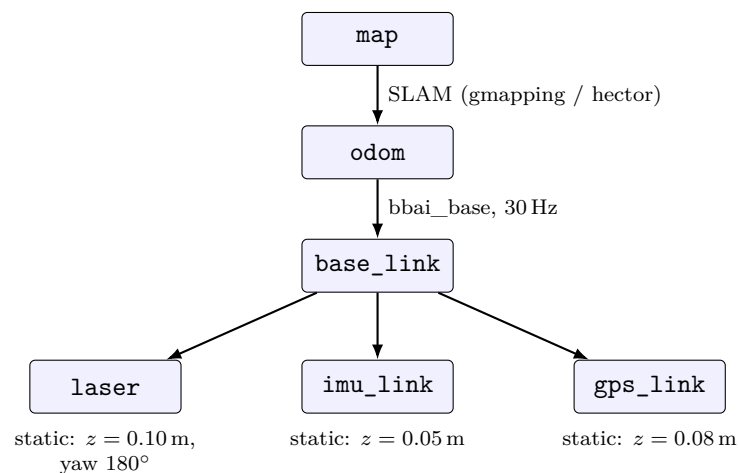


Figure 4.2: Transform tree

The three static transforms in `robot.launch` have zero horizontal offset; their positions are placeholders that have not been measured. The lidar transform has a yaw of 180° (π): a mount test showed that the A1’s zero bearing points at the robot’s rear (§10.5). The IMU and GPS transforms have zero rotation.

Each transform in the tree must have exactly one publisher. In particular, only one node may

publish odom→base_link: either bbai_base (publish_tf: true, the current setting) or an EKF, never both.

4.5 BRINGUP SEQUENCE

1. Board: `roslaunch bbai_base robot.launch`. This starts the master, `rplidar_ros`, `bbai_base` (which measures the gyro bias for 2s, so the robot must be still), `gps_node`, and the three static transforms. Arguments `lidar:=false` and `gps:=false` leave those devices out. `bbai_base` is respawned automatically 5s after it exits.
2. Board: `roslaunch bbai_teleop teleop.launch`.
3. Laptop: `~/bbai_slam/start_slam.sh [gmapping|hector|lidar_only]` (gmapping by default), which also opens RViz.
4. Laptop, optional: `roslaunch ~/bbai_slam/gps_fusion.launch`.
5. Board, optional: `roslaunch bbai_chase chase.launch` (dry run unless `dry_run:=false`).

The full command list is `docs/COMMANDS.md`.

4.6 BASE NODE

`base_node.py` runs a fixed-rate loop at 30 Hz. Each cycle it reads the IMU and the four encoders, integrates the pose (Chapter 5), publishes `/odom`, the `odom→base_link` transform, `/imu/data_raw`, `/imu/mag` and `/wheel_ticks`, and enforces the command timeout: if no `/cmd_vel` message has arrived for 0.5s, all motors are set to zero. Motors are also zeroed when the node exits.

With `mag_heading: true` it also publishes `/imu/data`: the same rates and accelerations as `/imu/data_raw` plus an orientation whose yaw comes from the calibrated magnetometer (§7.5). While the robot is parked it applies a zero-velocity update to the heading and the gyro bias (§7.3.3).

4.7 PS4 TELEOP

`ps4_teleop.py` reads the Linux joystick API directly (`/dev/input/js*`, 8-byte events of time, value, type and number) and looks the device up by name (`Wireless Controller`), skipping the controller’s separate motion-sensor device. It publishes `/joy` and, only while the deadman button is held, `/cmd_vel`. Publishing runs as a 20 Hz heartbeat that speeds up to at most 100 Hz while the sticks move, and all queued events are drained on each read so the commands never lag behind a backlog.

The stick values are $a \in [-1, 1]$ after dividing by 32767, with a dead zone $|a| < 0.08$ mapped to zero. Stick “up” is negative on the joystick API, so both axes are negated. In the repository version

Table 4.3: PS4 controller mapping (`hid-sony`, kernel 4.14)

Input	Index	Effect
L1 (hold)	button 4	Deadman. Nothing moves unless held; release sends one zero Twist
Left stick vertical	axis 1	<code>linear.x</code> , 0.3 m/s full scale (turbo 0.7 m/s)
Right stick horizontal	axis 3	<code>angular.z</code> (curve below)
R1 (with L1)	button 5	Turbo scaling
R2 (L1 released)	button 7	Arms the chase node

the command is linear in the stick,

$$v = -k_v a_1, \quad \omega = -k_\omega a_3, \quad (4.1)$$

with $(k_v, k_\omega) = (0.3, 1.5)$ normally and $(0.7, 8.0)$ in turbo.

In flux. On 2 October 2026 the turn axis was changed to a curve because 1.5 rad/s at full stick could not overcome skid-steer friction (§5.6). The installed version gives about 1.5 rad/s at half stick, 6 rad/s at full stick and 10 rad/s at full stick in turbo. It is installed on the board but not yet in the repository.

4.8 CHASE NODE

Chapter 12 covers the chase behaviour in full; this section summarizes the node.

`chase_node.py` runs a MobileNet-SSD spatial detection network on the OAK-D-Lite: the 300×300 RGB preview feeds the network on the camera’s VPU, and the stereo depth (aligned to RGB) gives each detection a 3-D position (x, z) in the camera frame, x to the right and z forward. The node keeps the most confident detection whose label is in `targets` and computes a bearing and range,

$$\beta = \text{atan2}(x, z), \quad \rho = \sqrt{x^2 + z^2}, \quad (4.2)$$

falling back to $\beta = (c_x - \frac{1}{2}) \text{HFOV}$ from the box centre $c_x \in [0, 1]$ when the box has no valid depth. Both are smoothed with an exponential moving average of weight $a = 0.5$. After at least 3 consecutive detections the command is

$$\omega = \begin{cases} \text{sat}_{\omega_{\max}}(-k_\beta \beta) & |\beta| > \beta_{\text{db}} \\ 0 & \text{otherwise} \end{cases} \quad (4.3)$$

$$v = \text{sat}_{[0, v_{\max}]}(k_\rho(\rho - \rho_f)) \cdot \text{sat}_{[0, 1]}\left(1 - \frac{|\beta|}{\beta_{\text{full}}}\right), \quad \rho > \rho_{\text{stop}}, \quad (4.4)$$

with $k_\beta = 2.5$, $\omega_{\max} = 2$ rad/s, $\beta_{\text{db}} = 5^\circ$, $k_\rho = 0.4$, $v_{\max} = 0.2$ m/s, follow distance $\rho_f = 0.9$ m, $\beta_{\text{full}} = 25^\circ$ and stop distance $\rho_{\text{stop}} = 0.5$ m. The second factor of v slows the robot while it is still turning toward the target. The robot never reverses.

Command arbitration. Both teleop and chase publish `/cmd_vel`, and `bbai_base` applies whichever arrives last. Conflicts are avoided by the buttons: chase only publishes while R2

is held and L1 is not, and teleop only publishes while L1 is held. Chase also stops if `/joy` goes quiet for 0.5 s or the target is lost for 0.5 s. In the default dry-run mode chase never publishes `/cmd_vel` at all, only `/chase/cmd_vel_preview`.

Chapter 5

DRIVE KINEMATICS AND WHEEL ODOMETRY

5.1 NOTATION

Table 5.1 lists the geometric parameters, with the values in `ros/bbai_base/config/base.yaml`.

Table 5.1: Geometry and drive parameters

Symbol	Meaning	Value	<code>base.yaml</code> key
r	wheel radius	0.0396 m	<code>wheel_radius</code>
b	track (left–right wheel centres)	0.120 m	<code>track_width</code>
L	wheelbase (front–back axles)	0.101 m	—
N	encoder counts per wheel revolution	15	<code>counts_per_rev</code>
Δt	odometry period	1/30 s	<code>rate</code>
$v_{\max}$	wheel speed at duty 1 (lifted)	1.2 m/s	<code>max_wheel_speed</code>
$d_{\max}$	duty limit	0.60	<code>max_duty</code>
$d_{\min}$	duty floor for any nonzero command	0.12	<code>min_duty</code>
T_{cmd}	command timeout	0.5 s	<code>cmd_timeout</code>

The pose in the `odom` frame is $\mathbf{p} = [x \ y \ \psi]^\top$. Subscript k denotes the k -th cycle of the 30 Hz loop, and $\Delta(\cdot)_k$ the change between cycles $k - 1$ and k .

5.2 FROM ENCODER COUNTS TO WHEEL TRAVEL

Let c_j be the raw count of encoder channel $j \in \{1, 2, 3, 4\}$ and $\sigma_j \in \{+1, -1\}$ its sign from Table 3.3. One count is an arc of

$$\delta = \frac{2\pi r}{N} = \frac{2\pi(0.0396)}{15} = 0.01659 \text{ m.} \quad (5.1)$$

Each side averages its two wheels,

$$s_L = \frac{\delta}{2}(\sigma_1 c_1 + \sigma_3 c_3), \quad s_R = \frac{\delta}{2}(\sigma_2 c_2 + \sigma_4 c_4), \quad (5.2)$$

and the travel of the body centre in one cycle is

$$d_k = \frac{1}{2}(\Delta s_{L,k} + \Delta s_{R,k}). \quad (5.3)$$

Averaging both wheels per side halves the effect of a single wheel slipping or being spun by hand: a spurious count on one wheel moves its side’s average by half a count.

Remark 5.1 (Quantization). With $N = 15$ the encoders are coarse. One count is 16.6 mm of wheel travel, so d_k is quantized in steps of $\delta/4 = 4.1$ mm. The velocity published on `/odom`, $v_k = d_k/\Delta t$, is therefore quantized in steps of $\delta/(4\Delta t) = 0.124$ m/s at 30 Hz: at a steady 0.3 m/s it alternates between about 2 and 3 levels from one message to the next. This is the same trade the thesis identifies for its motor encoders (the sample-rate remark in thesis §7.5): raising the loop rate makes the differenced velocity noisier for a fixed encoder resolution. Position, which is a sum of counts, is not affected; only the per-message velocity is. Anything that consumes the twist (the EKF in particular) should either use a longer differencing window or carry a velocity variance of at least $\delta^2/(192 \Delta t^2) \approx 1.3 \times 10^{-3} \text{ m}^2/\text{s}^2$, the variance of a uniform quantization error of width $\delta/(4\Delta t)$.

5.3 SKID-STEER KINEMATICS

For a differential-drive robot with no wheel slip the body speed and yaw rate are

$$v = \frac{1}{2}(v_R + v_L), \quad \omega = \frac{v_R - v_L}{b}. \quad (5.4)$$

A four-wheel skid-steer robot cannot turn without its wheels sliding sideways, because the front and back wheels on each side are fixed parallel at a distance L apart. The instantaneous centres of rotation of the left and right treads move outward, which is captured by an effective track χb with $\chi \geq 1$ [4, 5]:

$$\omega = \frac{v_R - v_L}{\chi b}. \quad (5.5)$$

χ depends on the ratio L/b (here 0.84, which is high), the floor surface, the load and the turn rate itself; values of 1.5 to 2 are typical. An encoder-only heading using (5.4) would overestimate every turn by the factor χ , and that factor would change between carpet and tile. This is why the base node takes its heading from the gyroscope (`use_gyro_heading: true`) and uses the encoders only for distance.

5.4 ODOMETRY INTEGRATION

The heading increment is

$$\Delta\psi_k = \begin{cases} (g_{z,k} - \hat{b}_g)\Delta t_k & \text{gyro heading (default)} \\ \frac{\Delta s_{R,k} - \Delta s_{L,k}}{b} & \text{encoder heading} \end{cases} \quad (5.6)$$

where g_z is the z gyroscope rate and $\hat{b}_g$ the bias estimate (§7.3). While the robot is parked, $\Delta\psi_k = 0$ (§7.3.3). The pose is advanced with the midpoint rule (second-order Runge–Kutta),

$$\begin{aligned}x_k &= x_{k-1} + d_k \cos\left(\psi_{k-1} + \frac{1}{2}\Delta\psi_k\right), \\y_k &= y_{k-1} + d_k \sin\left(\psi_{k-1} + \frac{1}{2}\Delta\psi_k\right), \\ \psi_k &= \text{wrap}(\psi_{k-1} + \Delta\psi_k),\end{aligned}\tag{5.7}$$

with $\text{wrap}(\alpha) = \text{atan2}(\sin \alpha, \cos \alpha) \in (-\pi, \pi]$. For motion on a circular arc the exact update is

$$x_k = x_{k-1} + \frac{d_k}{\Delta\psi_k} \left[\sin(\psi_{k-1} + \Delta\psi_k) - \sin \psi_{k-1} \right],\tag{5.8}$$

and similarly for y . Expanding both in $\Delta\psi_k$ shows that the midpoint rule differs from the exact arc only by a factor $1 - \Delta\psi_k^2/24$ on the step length. At the fastest turn the robot makes, 10 rad/s, $\Delta\psi_k = 0.33$ rad and the error is 0.5% of d_k , and in a pure spin $d_k \approx 0$ anyway, so (5.7) is adequate.

The message also carries the body velocity $v_k = d_k/\Delta t_k$ and $\omega_k = \Delta\psi_k/\Delta t_k$, and the same pose is broadcast as the transform `odom→base_link` with quaternion $\mathbf{q} = [0, 0, \sin \frac{\psi}{2}, \cos \frac{\psi}{2}]$.

5.4.1 Odometry uncertainty

Linearising (5.7) about the current estimate gives the usual dead-reckoning covariance propagation [6],

$$P_k = F_k P_{k-1} F_k^\top + G_k Q_k G_k^\top, \quad F_k = \begin{bmatrix} 1 & 0 & -d_k \sin \bar{\psi} \\ 0 & 1 & d_k \cos \bar{\psi} \\ 0 & 0 & 1 \end{bmatrix}, \quad G_k = \begin{bmatrix} \cos \bar{\psi} & 0 \\ \sin \bar{\psi} & 0 \\ 0 & 1 \end{bmatrix},\tag{5.9}$$

where $\bar{\psi} = \psi_{k-1} + \frac{1}{2}\Delta\psi_k$ and $Q_k = \text{diag}(\sigma_d^2, \sigma_\psi^2)$ is the variance of one step’s distance and heading increments. The position uncertainty grows without bound, and the heading term couples into it through the off-diagonal entries of F_k : a heading error $\delta\psi$ produces a lateral error $d\delta\psi$ after travelling d .

Remark 5.2 (The published covariance is constant). `base_node.py` publishes a fixed diagonal covariance, $(0.01, 0.01, 10^6, 10^6, 10^6, 0.05)$ for $(x, y, z, \phi, \theta, \psi)$ on both the pose and the twist. The 10^6 entries correctly mark the unobserved states. The pose entries, however, claim a 0.1 m standard deviation forever, while (5.9) grows. A filter that fuses the `/odom pose` as an absolute measurement will therefore trust it far too much after a few metres. The safe use is to fuse only the twist, or the pose in `robot_localization`’s `differential` mode. The GPS EKF does the former: it takes only v_x from `/odom` (§9.3).

5.5 FROM VELOCITY COMMAND TO MOTOR DUTY

A `/cmd_vel` message (v, ω) is turned into a desired speed for each side using the skid-steer model (5.5) with a commanded effective track χ_c ,

$$v_L = v - \frac{1}{2}\omega\chi_c, \quad v_R = v + \frac{1}{2}\omega\chi_c, \quad (5.10)$$

and each side speed into a PWM duty for both of that side's motors,

$$u = \frac{v_{\text{side}}}{v_{\text{max}}}, \quad D = \text{sat}_{d_{\text{max}}}(\text{sgn}(u)[d_{\text{min}} + (1 - d_{\text{min}})|u|]), \quad (D = 0 \text{ if } |u| < 10^{-3}), \quad (5.11)$$

multiplied by the motor's sign.

Effective track for commands. Originally χ_c was the geometric track $b = 0.12$ m. On the floor a command of 5.0 rad/s then produced only 1.31 rad/s as measured by the gyro, because the tyres scrub sideways and the robot turns as if its track were wider (§5.3). The ratio gives the effective track directly,

$$\chi_c = b \frac{\omega_{\text{cmd}}}{\omega_{\text{meas}}} = 0.12 \times \frac{5.0}{1.31} \approx 0.46 \text{ m}, \quad (5.12)$$

now the `cmd_turn_track` parameter in `base.yaml`, so that `/cmd_vel`'s ω approximately equals the real yaw rate on that floor. Odometry is unaffected: it still takes heading from the gyro.

In flux. `cmd_turn_track` was measured on one floor; carpet and hard floor will differ.

The offset d_{min} is a deadzone compensator: below about 8% duty the wheels do not turn at all (lifted test), so any nonzero command starts at 12%. This is the inverse of the deadzone nonlinearity the thesis models for its own motors (thesis §7.6.2). The control is open loop: duty sets voltage, and the speed that results depends on load and battery voltage. The encoders are used for odometry but not for speed feedback.

Table 5.2 evaluates (5.10)–(5.11) for the commands teleop sends, with the old and new χ_c . The duty limit of 0.6 is reached at $|u| = 0.545$, so the fastest either side can be commanded is about 0.65 m/s, and with $\chi_c = 0.46$ m the fastest commanded spin is $2 \times 0.65/0.46 \approx 2.8$ rad/s.

Table 5.2: Commanded duty (D_L, D_R) for typical commands

Command	$\chi_c = b = 0.12$ m (old)	$\chi_c = 0.46$ m (new)
Forward 0.3 m/s	0.34, 0.34	0.34, 0.34
Spin 1 rad/s	−0.16, 0.16	−0.29, 0.29
Spin 1.5 rad/s (old full stick)	−0.19, 0.19	−0.37, 0.37
Spin 6 rad/s (new full stick)	−0.38, 0.38	−0.60, 0.60 (limit)
Spin 10 rad/s (new turbo)	−0.56, 0.56	−0.60, 0.60 (limit)
Forward 0.3 m/s + 1.5 rad/s	0.27, 0.41	−0.15, 0.59

5.6 WHY SKID-STEER TURNING NEEDS SO MUCH POWER

Jason observed on 2 October 2026 that normal-speed turning would not start the robot spinning from rest and barely turned it while moving. A simple friction model explains this [7]. Let the robot weigh W , shared equally by the four wheels, which sit at $\pm L/2$ from the centre along the body and $\pm b/2$ across it. Spinning in place, each wheel must slide sideways, and the floor resists with a lateral friction force $\mu_\ell W/4$ acting at a lever arm of $L/2$. The total resisting moment is

$$M_r = 4 \cdot \mu_\ell \frac{W}{4} \cdot \frac{L}{2} = \frac{\mu_\ell W L}{2}. \quad (5.13)$$

The motors supply it with opposing tractive forces $\pm F$ per side at lever arm $b/2$, a moment of $F b$. Turning starts only when

$$F \geq \frac{\mu_\ell L}{2b} W \approx 0.42 \mu_\ell W. \quad (5.14)$$

Driving straight only has to overcome rolling resistance $f_r W$ with f_r of order 0.02–0.05, whereas lateral sliding friction μ_ℓ is of order 0.3–0.7 on household floors. A spin therefore needs roughly 3 to 15 times the tractive force of driving straight. With the old linear mapping, full stick asked for 1.5 rad/s, which with the geometric track is a duty of only 0.19 (Table 5.2), barely above the 0.12 floor; that is enough to keep a lifted wheel turning but not to make the tyres scrub. This motivated the new teleop turn curve (6 rad/s at full stick) and then the effective command track $\chi_c = 0.46$ m, which roughly doubles the duty of every turn command. A closed-loop yaw-rate or wheel-speed controller (Chapter 6) would make turn response independent of the floor surface.

Remark 5.3. (5.14) shows the turn force grows with L/b . Moving the wheels farther apart, or using wheels with less lateral grip, would reduce it.

Chapter 6

MOTOR CONTROL AND TRACKING LOOPS

This chapter follows a velocity command from `/cmd_vel` down to the voltage across each motor, models the motors, lists every feedback and tracking loop on the robot with its rate, and then proposes the two loops the robot is missing: a closed-loop wheel-speed controller and a yaw-rate controller. Sections marked *proposed* describe designs that are not yet in the code.

6.1 THE ACTUATOR CHAIN

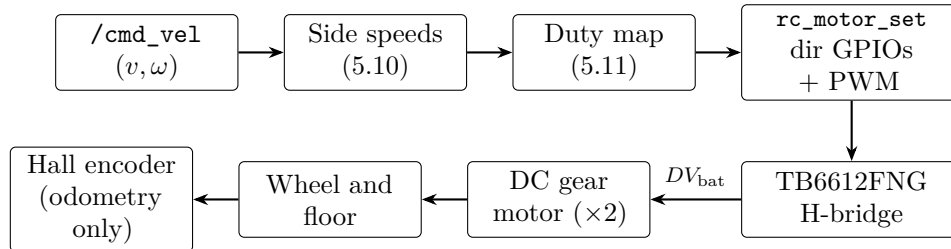


Figure 6.1: Path from a velocity command to the wheels (one side). There is no speed feedback today: the encoders feed odometry only.

Figure 6.1 shows the chain. `bbai_base` receives (v, ω) , splits it into side speeds (5.10), and maps each to a signed duty $D \in [-0.6, 0.6]$ with the deadzone offset (5.11). `librobotcontrol`’s `rc_motor_set(ch, D)` then

1. multiplies D by the channel polarity (the fork’s table is $\{1, -1, 1, -1\}$ for M1–M4, so the wiring of each motor’s leads does not matter to the caller),
2. sets the two direction inputs `IN1/IN2` of that H-bridge channel from the sign of D (for M1 one of these is driven by the PRU firmware of §3.5), and
3. writes $|D|$ to the channel’s PWM output at 25 kHz (`RC_MOTOR_DEFAULT_PWM_FREQ`).

The 25 kHz carrier is above hearing and far above the motor's electrical corner frequency R/L , so the motor current is essentially DC with a small ripple.

Drive and brake. The TB6612FNG works in *sign-magnitude* mode: during the on-part of each PWM period the bridge applies $\pm V_{\text{bat}}$ to the motor, and during the off-part it shorts both motor terminals together (short brake, the chip's behaviour when PWM is low with a direction set). The current recirculates through the low-side switches, so in continuous conduction the average terminal voltage is simply

$$\bar{V} = D V_{\text{bat}} - 2iR_{\text{on}}, \quad (6.1)$$

where $R_{\text{on}} \approx 0.5 \Omega$ is the on-resistance of one switch. Setting $D = 0$ (all four duty outputs zero) brakes the motors rather than letting them coast; `rc_motor_free_spin` would coast them, but the base node does not use it.

6.2 DC MOTOR MODEL

The thesis models its permanent-magnet DC motors with the standard electrical and mechanical equations (thesis §5.1). Written for one motor of this robot, with armature current i , motor-shaft speed Ω_m and gear ratio N (so the wheel turns at $\Omega = \Omega_m/N$),

$$L_a \frac{di}{dt} = \bar{V} - R_a i - K_e \Omega_m, \quad J_m \frac{d\Omega_m}{dt} = K_t i - b_m \Omega_m - \frac{\tau_\ell}{N\eta}, \quad (6.2)$$

with armature resistance R_a and inductance L_a , back-EMF constant K_e , torque constant K_t ($= K_e$ in SI units), viscous friction b_m , gearbox efficiency η , and load torque τ_ℓ at the wheel. The electrical time constant L_a/R_a is well under a millisecond for motors this size, so the current settles almost instantly and (6.2) reduces to a first-order model for the wheel speed. With $\tau_\ell = rF$ for a tractive force F at the tyre,

$$\tau_m \frac{d\Omega}{dt} + \Omega = \underbrace{\frac{K_t}{N(K_t K_e + R_a b_m)}}_{K_V} \bar{V} - \underbrace{\frac{R_a}{N^2 \eta (K_t K_e + R_a b_m)}}_{K_F} rF, \quad \tau_m = \frac{R_a J_{\text{eq}}}{K_t K_e + R_a b_m}, \quad (6.3)$$

where J_{eq} includes the motor rotor and the reflected inertia of the wheel and its share of the robot's mass.

What this says about open-loop duty control. In steady state the wheel speed is

$$v_{\text{wheel}} = r\Omega = rK_V(D V_{\text{bat}} - 2iR_{\text{on}}) - rK_F rF. \quad (6.4)$$

The duty map (5.11) assumes the first term alone with a fixed V_{bat} . Equation (6.4) shows two errors in that assumption:

- *Battery voltage.* A 2S LiPo falls from 8.4 V full to about 7.0 V near empty, so the same duty gives about 17% less speed by the end of a run.
- *Load.* Any force at the tyre subtracts speed in proportion to K_F . Driving straight on a level floor the load is small (rolling resistance), but in a skid-steer turn it is large: (5.14) says each side must push with about $0.42\mu_\ell W$ before the robot rotates at all. The second term then cancels most of the first, which is why low-duty turns stalled (§5.6).

The deadzone below about 8% duty is the same equation with Coulomb friction in the gearbox: the motor must make a minimum torque before it moves.

Identifying the model. K_V and τ_m can be measured with the wheels lifted (where $F \approx 0$) by stepping the duty and logging `/wheel_ticks`: the final speed divided by DV_{bat} gives K_V (the lifted test of 2026-10-01 gives about $0.4 \text{ m/s} / (0.3 \times 7.9 \text{ V}) \approx 0.17 \text{ m/s/V}$ at the wheel), and the time to 63% of it gives τ_m . Repeating the step on the floor in a spin gives the load term. Because of the coarse encoder (§6.4), the step response should be fitted to the accumulated count, $\int v dt$, rather than to a differentiated speed.

6.3 LOOPS ON THE ROBOT TODAY

Table 6.1 lists every periodic loop and tracking loop in the system, from the lowest level up.

Table 6.1: Periodic, feedback and tracking loops

Loop	Rate	Where	What it closes
PWM carrier	25 kHz	cape PWM	None; sets average voltage (6.1).
Encoder counting	per edge	eQEP $\times 3$, PRU	None; hardware counters.
Base loop	30 Hz	<code>bbai_base</code>	Reads encoders and IMU, integrates odometry (5.7), publishes <code>/odom</code> and <code>/imu/*</code> .
Command watchdog	30 Hz check, 0.5 s timeout	<code>bbai_base</code>	Stops the motors when <code>/cmd_vel</code> goes quiet.
Wheel speed	none		<i>Open loop</i> : duty from (5.11). Proposed in §6.5.
Gyro bias (ZUPT)	30 Hz while still	<code>bbai_base</code>	Tracks the gyro bias toward the measured rate when the robot is stationary (§7.3.3).
Heading	30 Hz	<code>bbai_base</code>	Complementary gyro-compass filter (§7.6).
Teleop	20 Hz idle, up to 100 Hz	<code>bbai_teleop</code>	Human in the loop.
Chase	15 Hz	<code>bbai_chase</code>	Bearing and range to a detected dog or person (Chapter 12).
Chase yaw rate	15 Hz	<code>bbai_chase</code>	Gyro PI loop on the requested turn rate (§12.4.2; PR #3, untested).
Detection	about 10 Hz	OAK-D-Lite	Runs on the camera; feeds the chase loop.
Global EKF	20 Hz	<code>ekf_gps</code>	Fuses odometry, IMU yaw and GPS (Chapter 9).
SLAM	per scan (5 Hz to 7 Hz)	<code>gmapping</code> / <code>hector</code>	Scan matching against the map (Chapter 10).

The only loops that close around the robot’s motion today are the human (teleop) and the chase controller, and both close around the motors through the open-loop duty map. Everything between the velocity command and the wheel is feed-forward.

6.4 ENCODER SPEED ESTIMATION

Any speed loop needs a speed measurement, and the encoders are coarse: 15 counts per wheel turn, $\delta = 2\pi r/15 = 16.6$ mm of travel per count. A side’s speed averaged over its two encoders and differenced over one 30 Hz sample is quantized in steps of

$$\Delta v = \frac{\delta/2}{\Delta t} = 0.25 \text{ m/s}, \quad (6.5)$$

which is most of the robot’s useful speed range. Two standard remedies, both from the thesis’s discussion of encoder sample rate (thesis §7.5):

1. *Fixed-window differencing.* Difference over the last M samples: $\hat{v}_k = (s_k - s_{k-M})/(M\Delta t)$, where s is the accumulated distance. The quantization step falls to $\Delta v/M$ and the estimate lags by $M\Delta t/2$. $M = 6$ (0.2 s) gives 0.04 m/s steps and a 0.1 s lag.
2. *Period measurement.* Time-stamp each count edge and use $\hat{v} = \delta/T_{\text{edge}}$. This is much better at low speed but needs edge timestamps, which the eQEP units can capture (their unit timer and capture registers) but librobotcontrol does not expose.

Option 1 needs no driver change and is what the proposed loop below assumes; its lag is part of the loop delay.

6.5 PROPOSED: WHEEL-SPEED LOOP

Proposed, not implemented. A per-side PI controller around the first-order model (6.3), with the current duty map as feed-forward:

$$\begin{aligned} e_k &= v_k^* - \hat{v}_k, \\ D_k &= \underbrace{D_{\text{ff}}(v_k^*)}_{(5.11)} + K_p e_k + I_k, \\ I_{k+1} &= I_k + K_i \Delta t e_k + K_{aw} (\text{sat}(D_k) - D_k), \end{aligned} \quad (6.6)$$

where sat is the ± 0.6 duty limit and the last term is back-calculation anti-windup: while the output is saturated, the integrator is bled toward the value that just reaches the limit, so it does not wind up during a stalled turn and overshoot when the robot breaks free. The feed-forward does most of the work on a level floor; the integrator supplies the extra duty a skid-steer turn needs (5.14) and compensates battery sag.

Gain selection. Linearising (6.3) about an operating point, duty to side speed is $G(s) = K_D/(\tau_m s + 1)$ with $K_D = rK_V V_{\text{bat}}$ (about 1.3 m/s per unit duty from the lifted test). With PI $C(s) = K_p + K_i/s$, choosing the PI zero on the motor pole, $K_i/K_p = 1/\tau_m$, cancels it and leaves the closed loop

$$\frac{\hat{v}}{v^*} = \frac{1}{\tau_c s + 1}, \quad \tau_c = \frac{\tau_m}{K_p K_D}. \quad (6.7)$$

The estimator lag $M\Delta t/2 = 0.1$ s and the 33 ms sample period act as a delay $\tau_d \approx 0.12$ s; for a phase margin of about 60° the crossover $1/\tau_c$ should be at most about $\pi/(6\tau_d) \approx 4.4$ rad/s, so $\tau_c \approx 0.25$ s. With an assumed $\tau_m = 0.1$ s (to be measured as in §6.2) that gives $K_p \approx 0.3$ and $K_i \approx 3$ per m/s, and $K_{aw} \approx K_i/K_p$.

Interaction with odometry. The loop changes nothing about how odometry is computed. It makes the commanded and actual speeds agree, which also makes the EKF’s process model, which assumes the robot does what it is told between measurements, more accurate.

6.6 PROPOSED: YAW-RATE LOOP

Proposed, not implemented. Even with each side tracking its speed, a skid-steer robot does not turn at $\omega = (v_R - v_L)/b$: the wheels slip sideways and the effective track χ of (5.5) is larger than b and depends on the floor. The gyro measures the true yaw rate directly, so an outer loop on yaw rate removes that uncertainty:

$$\Delta v_k = \frac{\chi_0}{2} \omega_k^* + K_\omega (\omega_k^* - \omega_{g,k}) + J_k, \quad J_{k+1} = J_k + K_{\omega i} \Delta t (\omega_k^* - \omega_{g,k}), \quad (6.8)$$

with $v_{L,R}^* = v \mp \Delta v$ passed to the speed loops, ω_g the bias-corrected gyro rate from the base node, and χ_0 a nominal effective track. The feed-forward part now exists: `cmd_turn_track` = 0.46 m ((5.12)) is exactly χ_0 , so the loop’s integrator only has to correct for how the floor differs from the one it was measured on. The gyro is sampled at the same 30 Hz but needs no differencing, so this loop has less delay than the speed loop and can run somewhat faster than it. The integrator J learns the floor-dependent χ ; it should be frozen whenever $|\omega^*|$ is small, so it does not fight the straight-line drift that the heading estimate already handles.

The chase node now implements a simpler form of this loop for its own turn commands, as a PI correction on ω ahead of the base node (§12.4.2, PR #3, untested). Moving it into the base would give teleop the same benefit.

6.7 IMPLEMENTATION NOTES

- Both loops belong in `bbai_base`, at the existing 30 Hz, so they share the encoder and gyro reads already there. Raising the base rate to 50 Hz would help the yaw loop more than the speed loop, whose limit is encoder resolution.

- The watchdog must reset both integrators when it stops the motors, and the feed-forward deadzone offset should stay so that small commands still move the robot before the integrator catches up.
- A parameter `closed_loop: false` would keep the present behaviour available for comparison.

Chapter 7

ATTITUDE AND HEADING

The robot drives on flat floors, so the navigation state needs only the yaw ψ from the IMU. Roll and pitch matter only for tilt-compensating the magnetometer. This chapter derives the gyroscope heading used by the odometry, its error growth, the accelerometer tilt, and the magnetometer compass.

7.1 SENSOR FRAME

`librobotcontrol` returns the gyroscope, accelerometer and magnetometer in one common frame: the AK8963 magnetometer inside the MPU-9250 has its x and y axes swapped and z reversed relative to the gyroscope, and the library undoes that (`mpu.c`, $m_x \leftarrow m'_y$, $m_y \leftarrow m'_x$, $m_z \leftarrow -m'_z$). The static transform `base_link`→`imu_link` has zero rotation, which assumes the cape's IMU axes coincide with the robot's x -forward, y -left, z -up axes. The yaw-rate sign has been confirmed in practice (gyro heading turns the right way in RViz and agrees with the compass over two turns); the x and y axes have not been checked separately.

7.2 SENSOR ERROR MODEL

Following the thesis (thesis §7.5, after Gebre-Egziabher [8] and Flenniken [9]), each inertial channel is modelled as

$$m = m_r + c_r + b_r + w_r, \quad (7.1)$$

where m_r is the true value, c_r a constant bias that changes at every power-up, b_r a walking bias and w_r white noise of variance σ_r^2 . The walking bias is a first-order Gauss–Markov process,

$$\dot{b}_r = -\frac{1}{\tau_r} b_r + w_{b_r}, \quad (7.2)$$

with w_{b_r} white of intensity $2\sigma_{b_r}^2/\tau_r$, so that $\sigma_{b_r}^2$ is its steady-state variance. The thesis characterised an MPU-class gyroscope this way, with Allan variance, and found $\tau_r \approx 1600$ s (thesis Table 7.2); the

same procedure can be reused on this IMU by logging `/imu/data_raw` at rest for an hour.

7.3 GYROSCOPE HEADING

7.3.1 Bias removal at start-up

At start the base node averages the z rate for $T_b = 2$ s with the robot stationary,

$$\hat{b}_g = \frac{1}{n} \sum_{i=1}^n g_{z,i}, \quad (7.3)$$

which estimates $c_r + b_r(0)$ in (7.1). This is the same start-up bias removal used on the TWIP (thesis §4.3). The corrected rate $g_z - \hat{b}_g$ is integrated in (5.6).

7.3.2 Heading error growth

The heading error after time t has three parts. Integrating white noise gives an angle random walk; an error $\delta b = \hat{b}_g - (c_r + b_r(0))$ in the start-up estimate gives a linear ramp; and the walking bias adds a slower term,

$$\delta\psi(t) = \underbrace{\int_0^t w_r ds}_{\sigma=N_g\sqrt{t}} + \underbrace{\delta b t}_{\text{start-up error}} + \int_0^t (b_r(s) - b_r(0)) ds. \quad (7.4)$$

The MPU-9250 datasheet gives a rate-noise density of $N_g = 0.01^\circ/\text{s}/\sqrt{\text{Hz}}$. The random-walk term is therefore $0.01^\circ\sqrt{t}/\text{s}$, under 0.1° after one minute. Averaging over $T_b = 2$ s leaves δb with a standard deviation of about $N_g/\sqrt{T_b} = 0.007^\circ/\text{s}$, which would give about $0.4^\circ/\text{min}$.

The measured drift at rest was $1.6^\circ/\text{min}$ ($0.027^\circ/\text{s}$), four times larger. The noise terms cannot account for it, so most of it is bias that moves after the start-up average: temperature change as the board and motor drivers warm up (the datasheet allows the zero-rate offset to move by up to $\pm 30^\circ/\text{s}$ over the full temperature range) and the walking bias. Over a 10-minute mapping run this is about 16° of heading error if nothing corrects it. Two practical consequences:

- Measuring the bias after the robot has been powered for a few minutes will reduce the error.
- The bias should keep being estimated while the robot runs. The base node now does this whenever the robot is parked (§7.3.3); the magnetometer can also bound the heading (§7.6).

7.3.3 Zero-velocity update

When the robot is parked its true yaw rate is zero, so whatever the gyroscope reads is bias. The base node uses this. Let t_m be the last time any wheel tick changed or a `/cmd_vel` command was

active. If $t - t_m > T_z = 0.5$ s the robot is declared stationary and, each cycle,

$$\hat{b}_{g,k} = \hat{b}_{g,k-1} + \alpha(g_{z,k} - \hat{b}_{g,k-1}), \quad \alpha = \min\left(1, \frac{\Delta t}{\tau_z}\right), \quad \Delta\psi_k = 0, \quad (7.5)$$

with $\tau_z = 20$ s. The first part is a first-order low-pass filter (time constant τ_z) that tracks the walking bias b_r of (7.2); the second freezes the heading. In Kalman filter terms this is a pseudo-measurement $\omega = 0$ with zero variance on the heading and a steady-state gain α on the bias [10]. After the change, heading drift while parked fell from about $1.5^\circ/\text{min}$ to zero (2 October 2026). Drift while driving is still governed by (7.4), but every stop now re-estimates the bias.

Remark 7.1. The stationary test uses the wheels and the command, not the gyroscope. If the robot is picked up and turned by hand, or pushed sideways with the wheels locked, the heading will not follow, and the bias filter will absorb part of the real rotation. Carry the robot level and still, or restart `bbai_base`, when moving it by hand.

7.4 TILT FROM THE ACCELEROMETER

The accelerometer measures specific force, which at rest is gravity in body axes. With roll ϕ and pitch θ , the thesis derives (thesis eq. 4.2, after Kuipers [11])

$$\phi = \text{atan2}(a_y, a_z), \quad \theta = \text{atan2}\left(-a_x, \sqrt{a_y^2 + a_z^2}\right). \quad (7.6)$$

This assumes no acceleration other than gravity, which is close enough for this robot’s low accelerations, and could be filtered with the complementary filter of the thesis (thesis eq. 4.4) if needed. The current software does not use roll and pitch.

7.5 MAGNETOMETER COMPASS

7.5.1 Measurement model

The magnetometer measures Earth’s field $\mathbf{h}$ rotated into the body frame, distorted by nearby material, plus noise [12]:

$$\mathbf{m} = S R^T \mathbf{h} + \mathbf{b}_{\text{hi}} + \mathbf{w}_m. \quad (7.7)$$

R is the body-to-world rotation. The hard-iron offset $\mathbf{b}_{\text{hi}}$ is a constant field carried with the robot: permanent magnets in the motors and encoder rings, and steady currents. The matrix S combines the sensor’s scale factors, axis misalignment and soft-iron effects, where nearby steel bends Earth’s field. On this robot the measured field magnitude was about $150 \mu\text{T}$ against Earth’s $50 \mu\text{T}$, so $\mathbf{b}_{\text{hi}}$ dominates.

7.5.2 Planar calibration

When the robot spins on a level floor, R is a rotation about z alone and the horizontal components (m_x, m_y) trace an ellipse; without soft-iron distortion it is a circle of radius H (the horizontal field) centred on (b_x, b_y) . The calibration routine (`mag_calibrate.py`) commands 5 rad/s until the gyroscope has seen two full turns (or 40 s pass), logging (m_x, m_y) from `/imu/mag`. A lower command stalls on the floor (§5.6).

Circle fit (used). The routine fits a circle by linear least squares (the Kåsa fit [13]): writing the circle as $m_x^2 + m_y^2 + Dm_x + Em_y + F = 0$, each sample gives one linear equation in (D, E, F) , and the normal equations

$$\begin{bmatrix} \sum m_x^2 & \sum m_x m_y & \sum m_x \\ \sum m_x m_y & \sum m_y^2 & \sum m_y \\ \sum m_x & \sum m_y & n \end{bmatrix} \begin{bmatrix} D \\ E \\ F \end{bmatrix} = - \begin{bmatrix} \sum m_x z \\ \sum m_y z \\ \sum z \end{bmatrix}, \quad z = m_x^2 + m_y^2, \quad (7.8)$$

are solved by Cramer’s rule. The hard-iron offset is the centre $(o_x, o_y) = (-D/2, -E/2)$ and the horizontal field is $H = \sqrt{o_x^2 + o_y^2 - F}$. The routine also unwraps the compass heading over the spin and compares its direction with the gyroscope’s to set `mag_sign` $\in \{+1, -1\}$. The result on 1 October 2026 was $(o_x, o_y) = (11.71, 66.61)$ μT , $s = +1$. A circle fit removes the hard iron only; if the residual (printed as “fit error”) grows, one of the estimators below adds soft-iron correction.

Min–max (offset and per-axis scale).

$$b_x = \frac{1}{2}(m_x^{\max} + m_x^{\min}), \quad s_x = \frac{\bar{\rho}}{\frac{1}{2}(m_x^{\max} - m_x^{\min})}, \quad (7.9)$$

and likewise for y , with $\bar{\rho}$ the mean of the two half-ranges. The corrected field is $m'_x = s_x(m_x - b_x)$, $m'_y = s_y(m_y - b_y)$. This handles hard iron and unequal axis gains but not rotation of the ellipse.

Ellipse fit (full planar soft iron). Fit the conic

$$Am_x^2 + Bm_x m_y + Cm_y^2 + Dm_x + Em_y = 1 \quad (7.10)$$

to all samples by linear least squares. The centre $\mathbf{b} = -\frac{1}{2}M^{-1}[D \ E]^T$ with $M = \begin{bmatrix} A & B/2 \\ B/2 & C \end{bmatrix}$ is the hard-iron offset, and the correction $\mathbf{m}' = M^{1/2}(\mathbf{m} - \mathbf{b})$ maps the ellipse back to a circle.

The routine prints the resulting lines for `base.yaml`. The check during calibration was that over the two-turn spin the compass heading advanced 745° while the gyroscope advanced 724° , against 720° actual. The gyroscope’s scale factor is therefore good to about 0.6% and the compass’s residual distortion is about 3.5%.

7.5.3 Heading from the field

On level ground, write the world (ENU) field's horizontal part in terms of the magnetic declination D (positive when magnetic north is east of true north): $\mathbf{h}_{\text{hor}} = H[\sin D, \cos D]^T$ in (east, north). With the robot at ENU yaw ψ , the body-frame components are

$$m'_x = H \sin(\psi + D), \quad m'_y = H \cos(\psi + D), \quad (7.11)$$

so

$$\psi = \text{atan2}(m'_x, m'_y) - D. \quad (7.12)$$

This is exactly what `base_node.py` computes, with $m' = s(m - o)$ from the circle fit: $\psi_m = s \text{atan2}(m_x - o_x, m_y - o_y) - D$. It publishes ψ_m as the orientation of `/imu/data` with variance 0.05 rad^2 ($\sigma \approx 13^\circ$) on yaw and 10^3 on roll and pitch (unknown), and the gyroscope rate with variance $1 \times 10^{-4} \text{ rad}^2/\text{s}^2$. Checking the two easy cases: facing east ($\psi = 0$) with $D = 0$, north is on the robot's left, so $m'_y = H$, $m'_x = 0$ and $\psi = 0$; facing north ($\psi = \pi/2$), north is straight ahead, $m'_x = H$ and $\psi = \pi/2$.

Two conventions are easy to confuse here. A compass heading χ_c is measured clockwise from north, while ROS yaw is counter-clockwise from east:

$$\chi_c = \frac{\pi}{2} - \psi. \quad (7.13)$$

The robot's declination is set to 2.5° east, computed from the NOAA World Magnetic Model [14] for the robot's home location. (The coordinates themselves are deliberately not recorded in this document or the repository.)

7.5.4 Tilt compensation

If the robot is not level, the levelled field is

$$\mathbf{m}_h = R_y(\theta)R_x(\phi)\mathbf{m}', \quad (7.14)$$

using the ZYX convention in which $R = R_z(\psi)R_y(\theta)R_x(\phi)$, and (7.12) is applied to $(m_{h,x}, m_{h,y})$. With ϕ and θ from (7.6). On a level floor $\mathbf{m}_h = \mathbf{m}'$. A 5° tilt with a magnetic inclination of about 60° (typical of the southern United States) produces up to about 9° of heading error, so compensation matters on ramps and lawns but not indoors.

7.5.5 Limits indoors

Indoors, steel in floors, appliances and wiring adds local fields that move the compass by tens of degrees as the robot drives. The calibration above removes only fields carried with the robot. The compass is therefore useful outdoors for initialising the GPS frame (§8.3) but should not be trusted as an absolute heading inside the house.

7.6 FUSING GYRO AND COMPASS

The gyroscope is precise over seconds and drifts over minutes; the compass is noisy and locally biased but does not drift. The thesis used exactly this structure for tilt (thesis eq. 4.4), and the heading version is

$$\psi_{k+1} = \psi_k + (g_{z,k} - \hat{b}_g)\Delta t + (1 - \lambda) \text{wrap}(\psi_{m,k} - \psi_k - (g_{z,k} - \hat{b}_g)\Delta t), \quad (7.15)$$

where ψ_m is (7.12). The wrap is needed because the two headings can differ across the $\pm\pi$ boundary. The filter has a crossover time constant $\tau = \lambda\Delta t/(1 - \lambda)$: with $\Delta t = 1/30$ s, $\lambda = 0.999$ gives $\tau \approx 33$ s. Higgins [15] shows this is the steady-state Kalman filter for a random-walk heading with white measurement noise, which is the form the EKF of Chapter 9 implements with time-varying gains. (7.15) is not currently running. The compass feeds only the GPS EKF on the laptop; the odometry heading that gmapping uses is gyroscope only, which is deliberate given the indoor limits above.

Chapter 8

GPS MEASUREMENT MODEL

8.1 NMEA TO NAVSATFIX

`gps_node.py` reads `/dev/ttyS2` one line at a time, drops any sentence whose checksum fails (the XOR of every character between \$ and *, compared with the two hex digits after *), and handles two sentence types.

GGA (position). Latitude and longitude arrive as `ddmm.mmmm` and `dddmm.mmmm`. With the integer degrees d and minutes m ,

$$\varphi = \pm \left(d + \frac{m}{60} \right), \quad (8.1)$$

negative for S and W. The altitude published is the ellipsoidal height $h = H_{\text{MSL}} + N$, where H_{MSL} is the antenna altitude above mean sea level (field 9) and N the geoid separation (field 11). A quality field of 0 publishes `STATUS_NO_FIX` with NaN coordinates; quality 2 (differential/SBAS) publishes `STATUS_SBAS_FIX`.

Position covariance. The receiver does not report an error estimate, only the horizontal dilution of precision (HDOP). The node uses the standard approximation [16]

$$\sigma_h = \text{HDOP} \cdot \sigma_{\text{UERE}}, \quad \Sigma_{\text{ENU}} = \text{diag}(\sigma_h^2, \sigma_h^2, 4\sigma_h^2), \quad (8.2)$$

with a user-equivalent range error $\sigma_{\text{UERE}} = 5$ m and the vertical taken as twice the horizontal. With a typical open-sky HDOP of 1 this is a 5 m standard deviation, and the EKF automatically trusts a poor fix less.

RMC (velocity). Speed over ground s (knots, times 0.514 444 m/s) and course over ground c (degrees clockwise from true north) become an ENU velocity,

$$v_E = s \sin c, \quad v_N = s \cos c, \quad (8.3)$$

published on `/gps/vel` only while there is a fix. The EKF does not currently use it.

8.2 GEODETIC TO LOCAL COORDINATES

The EKF works in metres, so latitude φ , longitude λ and height h must be turned into a local Cartesian frame. On the WGS-84 ellipsoid (semi-major axis $a = 6\,378\,137$ m, eccentricity $e^2 = 6.694\,379\,99 \times 10^{-3}$) the Earth-centred, Earth-fixed position is

$$\begin{aligned} X &= (R_N + h) \cos \varphi \cos \lambda, \\ Y &= (R_N + h) \cos \varphi \sin \lambda, \\ Z &= (R_N(1 - e^2) + h) \sin \varphi, \end{aligned} \quad R_N = \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi}}, \quad (8.4)$$

and the ENU offset from a reference point (φ_0, λ_0) with ECEF position $\mathbf{r}_0$ is

$$\begin{bmatrix} e \\ n \\ u \end{bmatrix} = \begin{bmatrix} -\sin \lambda_0 & \cos \lambda_0 & 0 \\ -\sin \varphi_0 \cos \lambda_0 & -\sin \varphi_0 \sin \lambda_0 & \cos \varphi_0 \\ \cos \varphi_0 \cos \lambda_0 & \cos \varphi_0 \sin \lambda_0 & \sin \varphi_0 \end{bmatrix} (\mathbf{r} - \mathbf{r}_0). \quad (8.5)$$

Over the few hundred metres a robot this size covers, (8.5) reduces to the flat-Earth approximation

$$e \approx (R_N + h) \cos \varphi_0 \Delta \lambda, \quad n \approx (R_M + h) \Delta \varphi, \quad R_M = \frac{a(1 - e^2)}{(1 - e^2 \sin^2 \varphi_0)^{3/2}}, \quad (8.6)$$

with angles in radians. At mid-latitudes one degree of latitude is about 111 km, so the EM-506's 5 m error is about 4.5×10^{-5} degrees.

8.3 THE NAVSAT TRANSFORM

robot_localization's `navsat_transform_node` [17] uses the Universal Transverse Mercator (UTM) projection instead of (8.5): each fix becomes a UTM easting and northing $\mathbf{u}$, which is locally an ENU frame rotated by the small grid convergence angle (under 2° inside a zone, ignored here). The node builds a fixed transform from UTM to the EKF's world frame (`gps_map`) once, at the *datum*:

1. With `wait_for_datum: false`, the datum is the first fix that arrives (after a 2 s delay), together with the robot's pose in the EKF output $\mathbf{p}_0 = (x_0, y_0, \psi_0)$ and the IMU heading $\psi_{\text{imu},0}$ at that moment.
2. The robot's heading in UTM is $\psi_{\text{utm},0} = \psi_{\text{imu},0} + D_{\text{nst}} + \psi_{\text{off}}$. On this robot the declination is already applied on the board and `/imu/data` is ENU yaw, so `magnetic_declination_radians` $D_{\text{nst}} = 0$ and `yaw_offset` $\psi_{\text{off}} = 0$.
3. The transform maps a UTM point to the world frame by undoing the robot's UTM pose at

the datum and applying its world pose:

$$\mathbf{x}_w = R(\psi_0) R(\psi_{\text{utm},0})^\top (\mathbf{u} - \mathbf{u}_0) + \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}, \quad (8.7)$$

where $R(\cdot)$ is the planar rotation matrix and $\mathbf{u}_0$ the datum's UTM position (offset by the antenna lever arm, here zero).

Every later fix is pushed through (8.7) and published on `/odometry/gps` with the covariance (8.2) rotated by the same angle. That message is what the EKF fuses. The node also runs the inverse transform on the EKF output and publishes `/gps/filtered`, the fused position as latitude and longitude. `zero_altitude` is set, so height is ignored.

Remark 8.1 (The datum heading matters). Equation (8.7) rotates every GPS fix by the compass heading taken at a single instant. A heading error $\delta\psi$ at the datum rotates the whole GPS track about the starting point, so a fix ℓ metres away lands $\ell\delta\psi$ off to the side: 10° of compass error puts a point 20 m away 3.5 m off. The datum should therefore be taken outdoors, away from steel, with the robot still.

Chapter 9

SENSOR FUSION: THE GPS EKF

The global estimate combines wheel odometry, the gyroscope, the compass and GPS in an extended Kalman filter (EKF) from the `robot_localization` package [17], running on the laptop as node `ekf_gps` (`laptop/bbai_slam/gps_fusion.launch` and `gps_ekf.yaml`). This chapter states the filter’s model, the measurement model of each sensor, and how the result relates to the SLAM map.

9.1 THE DISCRETE EKF

The thesis gives the linear discrete Kalman filter in predictor–corrector form (thesis §7.7, after Kalman [18] and Simon [19]). The EKF is the same recursion with the state transition f and measurement h nonlinear and their Jacobians used for the covariance:

$$\hat{x}_k^- = f(\hat{x}_{k-1}^+, \Delta t_k), \quad P_k^- = F_k P_{k-1}^+ F_k^\top + Q_k \Delta t_k, \quad (9.1)$$

$$\nu_k = z_k - H_k \hat{x}_k^-, \quad S_k = H_k P_k^- H_k^\top + R_k, \quad (9.2)$$

$$K_k = P_k^- H_k^\top S_k^{-1}, \quad \hat{x}_k^+ = \hat{x}_k^- + K_k \nu_k, \quad (9.3)$$

$$P_k^+ = (I - K_k H_k) P_k^- (I - K_k H_k)^\top + K_k R_k K_k^\top, \quad (9.4)$$

with $F_k = \partial f / \partial \mathbf{x}$ at $\hat{x}_{k-1}^+$. The covariance update (9.4) is the Joseph form, the same one the thesis uses, which keeps P symmetric and positive definite under round-off. Angular components of the innovation are wrapped to $(-\pi, \pi]$. `robot_localization` runs the prediction at a fixed 20 Hz (**frequency**) and applies each sensor message as it arrives, after predicting forward to the message’s time stamp. Measurements may update any subset of the state; H_k is then the rows of the identity that select the components marked **true** in that sensor’s configuration vector.

9.2 STATE AND PROCESS MODEL

`robot_localization` always carries a 15-element state,

$$\mathbf{x} = [x \ y \ z \ \phi \ \theta \ \psi \ \dot{x}_b \ \dot{y}_b \ \dot{z}_b \ \dot{\phi} \ \dot{\theta} \ \dot{\psi} \ \ddot{x}_b \ \ddot{y}_b \ \ddot{z}_b]^\top, \quad (9.5)$$

pose in the world frame and velocities and accelerations in the body frame. With `two_d_mode: true`, z , ϕ , θ and their derivatives are fixed at zero, leaving the planar state $[x, y, \psi, v_x, v_y, \omega, a_x, a_y]$. The process model is the “omnidirectional” constant-acceleration model:

$$\begin{aligned} x_k &= x_{k-1} + (v_x \cos \psi - v_y \sin \psi) \Delta t + \frac{1}{2} (a_x \cos \psi - a_y \sin \psi) \Delta t^2, \\ y_k &= y_{k-1} + (v_x \sin \psi + v_y \cos \psi) \Delta t + \frac{1}{2} (a_x \sin \psi + a_y \cos \psi) \Delta t^2, \\ \psi_k &= \psi_{k-1} + \omega \Delta t, \\ v_{x,k} &= v_{x,k-1} + a_x \Delta t, \quad v_{y,k} = v_{y,k-1} + a_y \Delta t, \quad \omega_k = \omega_{k-1}, \end{aligned} \tag{9.6}$$

with accelerations held constant between updates. Its Jacobian has the familiar dead-reckoning entries $\partial x / \partial \psi = -(v_x \sin \psi + v_y \cos \psi) \Delta t + \dots$ and $\partial y / \partial \psi = (v_x \cos \psi - v_y \sin \psi) \Delta t + \dots$, the same coupling as (5.9). The process noise Q is the package default (a diagonal of small values, 0.05 on position and 0.06 on yaw per second); it has not been tuned for this robot.

9.3 MEASUREMENTS

Table 9.1 lists exactly what is fused. Each input contributes only the components marked, with the covariance its publisher reports.

Table 9.1: EKF inputs (`gps_ekf.yaml`)

Input	Topic	Fused	Variance
odom0	/odom	v_x	$0.01 \text{ m}^2/\text{s}^2$ ($\sigma = 0.1 \text{ m/s}$)
imu0	/imu/data	ψ, ω	ψ : 0.05 rad^2 ($\sigma \approx 13^\circ$); ω : $1 \times 10^{-4} \text{ rad}^2/\text{s}^2$
odom1	/odometry/gps	x, y	$(\text{HDOP} \cdot 5 \text{ m})^2$

Written out, the measurement functions are linear selections,

$$h_{\text{odom}}(\mathbf{x}) = v_x, \quad h_{\text{imu}}(\mathbf{x}) = \begin{bmatrix} \psi \\ \omega \end{bmatrix}, \quad h_{\text{gps}}(\mathbf{x}) = \begin{bmatrix} x \\ y \end{bmatrix}, \tag{9.7}$$

so all of the nonlinearity is in the process model.

Design choices.

- *Odometry contributes speed only.* The odometry pose and its yaw are not fused. The pose covariance from `bbai_base` is constant rather than growing (§5.4), so fusing it would over-trust it; and odometry yaw is the integrated gyroscope, which would double-count the gyroscope rate already fused from `imu0`. v_y is not fused either; the model has no constraint that keeps it at zero, so it is held small only by the process noise. Fusing $v_y = 0$ (a nonholonomic pseudo-measurement) would tighten the filter.
- *Heading is absolute from the compass, rate from the gyroscope.* The filter is then exactly the

Kalman form of the complementary filter (7.15): the gyroscope carries the heading between compass updates, and the compass bounds its drift. With variances of 1×10^{-4} on ω and 0.05 on ψ the compass corrects the heading slowly.

- *The gyroscope bias is not a state.* robot_localization has no bias states, so a constant rate error appears as a heading ramp that the compass pulls back. The bias estimate comes from the base node’s zero-velocity update instead (§7.3.3).
- *No outlier gating.* The Mahalanobis thresholds (*_rejection_threshold) are left at their default of infinity, so a GPS multipath jump is accepted at its stated covariance. Setting odom1_pose_rejection_threshold to about 3–5 (rejecting innovations with $\nu^T S^{-1} \nu$ above that many standard deviations squared) would guard against it.

9.4 OBSERVABILITY AND BEHAVIOUR WITHOUT A FIX

With a GPS fix the filter observes x , y and ψ directly and the covariance stays bounded. Without one (always, indoors), x and y are not observed: the filter integrates the odometry speed along the compass-and-gyro heading, and P_{xx} and P_{yy} grow as in (5.9). That is the “coasting” behaviour: /odometry/global is then a dead-reckoning estimate whose heading is anchored to the compass rather than to the gyro’s start-up direction. When fixes return, the first few updates pull the estimate onto the GPS track with a jump proportional to the drift accumulated.

9.5 RELATION TO THE SLAM MAP

The EKF publishes no transform (publish_tf: false). Its output lives in its own world frame gps_map, while gmapping owns map→odom. The two estimates run side by side:

	SLAM (gmapping)	GPS EKF
World frame	map	gps_map
Absolute reference	the lidar map	GPS and the compass
Works indoors	yes	coasts only
Publishes TF	map→odom	none
Consumers	RViz, map_saver	topic only

Both start from the same odometry, so for short runs they agree up to a fixed rotation and translation. This keeps the two from fighting, but nothing yet relates map to gps_map or to latitude and longitude.

Aligning the frames. If the robot drives a path outdoors where both are valid, the rigid transform between them follows from the paired positions $(\mathbf{a}_i, \mathbf{g}_i)$ in map and gps_map by least squares [20]:

$$\theta^* = \text{atan2}\left(\sum_i \bar{\mathbf{g}}_i \times \bar{\mathbf{a}}_i, \sum_i \bar{\mathbf{g}}_i \cdot \bar{\mathbf{a}}_i\right), \quad \mathbf{t}^* = \bar{\mathbf{a}} - R(\theta^*)\bar{\mathbf{g}}, \quad (9.8)$$

where $\bar{\mathbf{a}}, \bar{\mathbf{g}}$ are the means and $\bar{\mathbf{a}}_i, \bar{\mathbf{g}}_i$ the centred points ($\times$ is the planar cross product $g_x a_y - g_y a_x$). Publishing $(\theta^*, \mathbf{t}^*)$ as a static transform `map→gps_map` would put GPS fixes on the lidar map. A fuller design, for when the robot navigates on a saved map, replaces gmapping with a localiser (AMCL or slam_toolbox in localization mode) and a second, map-frame EKF that owns `map→odom` and fuses both the localiser's pose and GPS [17].

Chapter 10

MAPPING: LIDAR SLAM

The map is built on the laptop from the board’s `/scan` and the `odom→base_link` transform. Two SLAM packages are set up: `gmapping`, a Rao–Blackwellized particle filter that uses odometry and is the default, and `hector_mapping`, a scan matcher that can run on the lidar alone.

10.1 OCCUPANCY GRID

Both build a 2-D occupancy grid with 5 cm cells. Each cell m_i holds the log-odds of being occupied,

$$\ell_i = \log \frac{p(m_i)}{1 - p(m_i)}, \quad (10.1)$$

and every scan updates the cells along each beam by the inverse sensor model [6]:

$$\ell_{i,t} = \ell_{i,t-1} + \log \frac{p(m_i \mid z_t, x_t)}{1 - p(m_i \mid z_t, x_t)} - \ell_0. \quad (10.2)$$

Cells a beam passes through become more likely free, and the cell where it ends more likely occupied. Because the update is additive, a scan registered at the wrong pose does not erase the old walls; it adds a second, rotated or shifted copy. That is why a pose error shows up as a “jumbled” map rather than a moved one.

10.2 GMAPPING

10.2.1 Rao–Blackwellized particle filter

SLAM estimates the joint posterior of the robot’s path $x_{1:t}$ and the map m , given scans $z_{1:t}$ and odometry $u_{1:t-1}$. `gmapping` [21] factors it as

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t-1}) = p(m \mid x_{1:t}, z_{1:t}) p(x_{1:t} \mid z_{1:t}, u_{1:t-1}). \quad (10.3)$$

Given a path, the map follows by (10.2), so only the path needs a particle filter; each of the M particles carries one path hypothesis and its own map [22, 23]. Per accepted scan, each particle i :

1. **Predicts** with the odometry motion model: the change in `odom→base_link` since the last update, split into a translation Δ_ρ and a rotation Δ_ψ , is perturbed by Gaussian noise with standard deviations

$$\sigma_\rho = s_{rr} |\Delta_\rho| + s_{rt} |\Delta_\psi|, \quad \sigma_\psi = s_{tr} |\Delta_\rho| + s_{tt} |\Delta_\psi|, \quad (10.4)$$

(parameters `srr`, `srt`, `str`, `stt`).

2. **Scan-matches** the new scan against its own map, starting from the predicted pose x' , by hill-climbing in steps of `lstep` (m) and `astep` (rad) to maximise a likelihood-field score

$$s(x) = \sum_j \exp\left(-\frac{d_j(x)^2}{\sigma}\right), \quad (10.5)$$

where $d_j(x)$ is the distance from the endpoint of beam j (projected from pose x) to the nearest occupied cell, and σ is the parameter `sigma`. Only beams shorter than `maxUrange` are used.

3. **Samples** its new pose from a Gaussian fitted around the best match $\hat{x}^*$ (the “improved proposal”), which combines the scan match with (10.4). If the best score is below `minimumScore`, the match is considered unreliable and the particle falls back to the odometry prediction x' .
4. **Reweights**: $w_t^{(i)} \propto w_{t-1}^{(i)} \eta^{(i)}$, where $\eta^{(i)} = \int p(z_t | m^{(i)}, x) p(x | x_{t-1}^{(i)}, u_{t-1}) dx$ is approximated from the same samples.

When the effective sample size

$$N_{\text{eff}} = \frac{1}{\sum_i (\tilde{w}^{(i)})^2} \quad (10.6)$$

of the normalised weights $\tilde{w}$ falls below `resampleThreshold` · M , particles are resampled. The best particle’s map is published on `/map` every `map_update_interval` seconds, and its pose sets the transform `map→odom` such that

$$T_{\text{map} \leftarrow \text{base_link}} = T_{\text{map} \leftarrow \text{odom}} T_{\text{odom} \leftarrow \text{base_link}}. \quad (10.7)$$

That composition is how the SLAM pose and the odometry pose integrate: the odometry provides a smooth, 30 Hz pose that drifts, and SLAM publishes the slowly changing correction that cancels the drift.

10.2.2 When scans are processed

A scan is processed only if the robot has moved at least `linearUpdate` metres or turned `angularUpdate` radians according to odometry since the last processed scan, or, if `temporalUpdate` is positive, if that many seconds have passed.

10.2.3 Parameters

Table 10.1: gmapping parameters (`laptop/bbai_slam/gmapping_bbai.launch`)

Parameter	1 Oct	2 Oct	Effect
particles M	50	40	path hypotheses
temporalUpdate	2.0 s	off (−1)	process scans while parked
linearUpdate / angularUpdate	0.10 / 0.10	0.10 / 0.10	motion between scans (m / rad)
minimumScore	0	100	reject weak scan matches
srr, srt, str, stt	0.1, 0.2, 0.1, 0.2 [†]	0.05 each	odometry noise (10.4)
sigma, lstep, astep	0.05 [†]	0.05	scan-match kernel and steps
maxUrange / maxRange	8 / 12 m	6 / 12 m	usable / maximum beam length
map_update_interval	2.0 s	1.0 s	map publishing
resampleThreshold	0.5 [†]	0.5	resample when $N_{\text{eff}} < M/2$
delta	0.05 m	0.05 m	cell size
transform_publish_period	0.05 s [†]	0.05 s	map→odom rate

[†] package default, not set in the launch file

In flux. The 2 October values were still being tested when this was written.

10.3 HECTOR MAPPING

hector_mapping [24] has no particles and no odometry model. It aligns each scan directly to the map built so far. With the scan’s beam endpoints $\mathbf{s}_j$ in the robot frame and the pose $\boldsymbol{\xi} = (p_x, p_y, \psi)$, the endpoints in the map are

$$\mathbf{S}_j(\boldsymbol{\xi}) = R(\psi)\mathbf{s}_j + \begin{bmatrix} p_x \\ p_y \end{bmatrix}, \quad (10.8)$$

and the pose minimises

$$\boldsymbol{\xi}^* = \arg \min_{\boldsymbol{\xi}} \sum_j [1 - M(\mathbf{S}_j(\boldsymbol{\xi}))]^2, \quad (10.9)$$

where $M(\cdot) \in [0, 1]$ is the occupancy value bilinearly interpolated between grid cells. One Gauss–Newton step from the current estimate is

$$\Delta \boldsymbol{\xi} = H^{-1} \sum_j \left[\nabla M(\mathbf{S}_j) \frac{\partial \mathbf{S}_j}{\partial \boldsymbol{\xi}} \right]^T [1 - M(\mathbf{S}_j)], \quad H = \sum_j \left[\nabla M \frac{\partial \mathbf{S}_j}{\partial \boldsymbol{\xi}} \right]^T \left[\nabla M \frac{\partial \mathbf{S}_j}{\partial \boldsymbol{\xi}} \right], \quad (10.10)$$

with

$$\frac{\partial \mathbf{S}_j}{\partial \boldsymbol{\xi}} = \begin{bmatrix} 1 & 0 & -\sin \psi s_{j,x} - \cos \psi s_{j,y} \\ 0 & 1 & \cos \psi s_{j,x} - \sin \psi s_{j,y} \end{bmatrix}. \quad (10.11)$$

The match is run coarse-to-fine on a pyramid of maps (2 levels here) to widen its basin of convergence. Hector converges only if the true pose is within about one coarse cell and a few degrees of the previous estimate, so it fails when the robot turns faster than the scan rate can follow (§10.4). On this robot it is the fallback for mapping without odometry (`lidar_only_bbai.launch`), for example when carrying the robot by hand.

10.4 SCAN RATE AND ROTATION

The A1 delivers about 7.2 scans per second, so between scans, and during the 0.14s it takes to sweep one scan, the robot turns by

$$\Delta\psi_{\text{scan}} = \frac{\omega}{f_{\text{scan}}}. \quad (10.12)$$

Turn rate	0.5 rad/s	1.5 rad/s	6 rad/s
Rotation per scan	4°	12°	48°

Neither SLAM package corrects the distortion within a scan (each beam is treated as if taken at the scan’s time stamp), so a scan taken while turning at 1.5 rad/s is itself bent by up to 12° from its first beam to its last. gmapping’s `angularUpdate` of 0.1 rad (5.7°) also means that at 1.5 rad/s nearly every scan is processed while bent. Mapping is most reliable with turns below about 0.5 rad/s. The new teleop curve keeps half stick at 1.5 rad/s; turning gently (well under half stick) while mapping is better still.

10.5 DIAGNOSING THE SLAM DRIFT OF 1 OCTOBER

Jason observed that gmapping made a good first map, lost track once the robot moved, and, when left stationary, slowly rotated the map into a jumble. Each symptom has a distinct likely cause.

Map rotating while parked. With `temporalUpdate` at 2s, gmapping processed a scan every two seconds even when the robot did not move. The gyroscope heading drifted about 1.6°/min while parked (§7.3), so each new scan arrived with a slightly rotated odometry pose. The scan matcher absorbs a small rotation, but with only about 360 beams per scan and a cell size of 5 cm, its correction is imperfect, and the residual is written into the map by (10.2). Repeated every two seconds this smears the walls into a rotating blur. Two changes address it: `temporalUpdate = -1`, so a parked robot adds no scans, and the zero-velocity update (§7.3.3), which removes the heading drift while parked. Parked drift measured afterwards was zero.

Losing track while driving. Several causes can each produce this, and they are best ruled out in order:

1. **Lidar mounting yaw (found and fixed).** The static transform `base_link→laser` had zero yaw. `rplidar_ros` reports angle zero along the A1’s own zero direction, which on this robot points backward. With the yaw off by 180°, a spin in place still looks consistent (the scan rotates in the same direction either way), but driving forward makes the walls appear to approach from behind: odometry and scan disagree on every metre travelled and the map breaks as soon as the robot drives, which is exactly the symptom. `lidar_mount_test.py` points the same way (2 October 2026): driving forward shortened the ranges most at bearing

180°. The transform now has yaw π in `robot.launch` and in the laptop's `lidar_only-bbai.launch`. The turn checks in the same test are *unverified*: during them only one wheel was seen spinning and the robot did not actually turn, so the reported scan rotation and the small `map`→`odom` correction across $\pm 90^\circ$ turns need repeating with the robot really turning. The lidar's position is still assumed to be at the robot's centre until it is measured.

2. **Turning too fast.** See §10.4.
3. **Weak scan matches.** At the A1's range and with about 360 beams, narrow corridors and open rooms give the matcher little to hold on to. `minimumScore` now rejects weak matches in favour of odometry, which is reliable over short distances after the calibration (0.914 m measured for 0.914 m). `maxUrange` was reduced to 6 m because the A1's error grows with range.
4. **Odometry noise model.** With `srr-stt` at their defaults (0.1–0.2), particles spread far more than the calibrated odometry warrants, and with only 40–50 particles the filter can lock onto a wrong hypothesis. They are now 0.05.
5. **Clocks.** Scans and transforms are stamped by the board's clock, which has no battery-backed time source and is not synchronised to the laptop. gmapping looks up odometry at each scan's stamp, which both come from the board, so this is not expected to cause the drift, but running `chrony` on both machines removes a class of TF extrapolation warnings.

In flux. Longer mapping runs with the yaw fix have not been tested yet. `reset_map.sh` now waits for the old gmapping node to exit before relaunching.

Chapter 11

OAK-D-LITE DEPTH CAMERA

The OAK-D-Lite is the robot’s only camera and its only sensor that can tell a dog from a chair. It is a *stereo* depth camera, not a time-of-flight (ToF) camera: it measures depth by matching features between two image sensors a known distance apart, rather than by timing emitted light. The practical differences are that it needs texture in the scene (a blank wall gives no depth), it works outdoors in sunlight where many ToF sensors saturate, and its depth error grows with the square of range (11.6) rather than staying roughly constant. This chapter describes the hardware, the geometry that turns pixels into metres, the on-camera neural network, and the calibration state.

11.1 HARDWARE

Table 11.1: OAK-D-Lite specifications (Luxonis datasheet)

Item	Specification
Processor	Intel Movidius Myriad X VPU: 16 SHAVE vector cores, neural compute engine, hardware stereo-depth and warp engines; about 4 TOPS total, 1.4 TOPS for neural networks
Colour camera	Sony IMX214, 13 MP (4208×3120), rolling shutter, autofocus (8 cm to infinity), DFOV/H-FOV/VFOV 81°/69°/54°
Stereo pair	2× OmniVision OV7251, 640×480, global shutter, fixed focus, DFOV/HFOV/VFOV 86°/73°/58°
Stereo baseline	75 mm nominal; 74.3 mm from this unit’s calibration
Depth range	about 0.2 m (extended disparity) or 0.35 m (standard) to over 10 m; useful to about 8 m
Interface	USB-C, USB 3.2 Gen 1 capable; on the BeagleBone it runs at USB 2
Power	4 W to 6 W typical at 5 V, up to about 1.5 A peaks; powered from the USB port only
Size and mass	91 mm×28 mm×17.5 mm, 61 g

Table 11.1 gives the specifications. All image processing, stereo matching and neural-network inference run on the Myriad X; the BeagleBone receives only detection results (label, confidence, bounding box and 3-D position), so the camera adds almost no CPU load to the board. Its cost is power: at 0.8 A to 1.5 A it is the largest single load on the cape’s 5 V rail (§2.5).

Software. The board runs `dephai 2.19.1` for Python 3.7 (the newest wheel that installs on the board’s Debian). At USB 2 the link carries the detections and a preview stream easily; full-resolution video would not fit.

11.2 PINHOLE CAMERA MODEL

Each sensor is modelled as a pinhole camera with lens distortion removed by the calibration. A point $\mathbf{P} = (X, Y, Z)$ in the camera frame (X right, Y down, Z forward along the optical axis) projects to pixel (u, v) by

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z} \underbrace{\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}}_K \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}, \quad (11.1)$$

with focal lengths f_x, f_y in pixels and principal point (c_x, c_y) . For an image W pixels wide with horizontal field of view θ_h ,

$$f_x = \frac{W/2}{\tan(\theta_h/2)}. \quad (11.2)$$

For the mono cameras at $W = 640$ and the datasheet $\theta_h = 73^\circ$ this is $f \approx 432$ px; the calibration measured $f \approx 450$ px (Table 11.3), and the calibrated value is used from here on. Inverting (11.1) at a known depth Z gives the lateral position

$$X = \frac{(u - c_x) Z}{f_x}, \quad Y = \frac{(v - c_y) Z}{f_y}. \quad (11.3)$$

11.3 STEREO DEPTH

Rectification. Using the calibrated intrinsics and the rotation and translation between the two mono cameras, the Myriad’s warp engine remaps both images so that their image planes are coplanar and their rows are aligned. After rectification a point appears on the *same row* in both images, and the search for its match is one-dimensional along that row. The calibration’s epipolar error (how far matching points stray from a common row) measures how well this worked; this unit’s is 0.36 px.

Disparity and depth. If a point appears at column u_ℓ in the left image and u_r in the right, its disparity is $d = u_\ell - u_r$, and similar triangles give

$$Z = \frac{fB}{d}, \quad (11.4)$$

with baseline $B = 0.0743$ m. On this camera $fB \approx 450 \times 0.0743 = 33.4$ px m. The matcher is a census-transform cost with semi-global aggregation [25], run in hardware over a search range of 96

disparities (0–95), or 191 with extended disparity. The largest disparity sets the minimum depth,

$$Z_{\min} = \frac{fB}{d_{\max}} = \frac{33.4}{95} \approx 0.35 \text{ m} \quad (\text{standard}), \quad \frac{33.4}{190} \approx 0.18 \text{ m} \quad (\text{extended}). \quad (11.5)$$

The chase pipeline uses the standard range, so a dog closer than about 0.35 m has no valid depth and falls back to bearing-only (§12.3). The 0.5 m stop distance keeps the robot outside that zone.

Depth resolution. Differentiating (11.4), a disparity error δd gives a depth error

$$\delta Z = \frac{Z^2}{fB} \delta d. \quad (11.6)$$

Table 11.2 evaluates it for one whole pixel of disparity. With sub-pixel interpolation (3 fractional bits, so steps of 1/8 px) the quantization is eight times finer, though real matching noise is typically a few tenths of a pixel.

Table 11.2: Stereo depth resolution, $fB = 33.4 \text{ px m}$, $\delta d = 1 \text{ px}$

Range Z	Disparity d (px)	δZ
0.5 m	66.8	0.007 m
1 m	33.4	0.030 m
2 m	16.7	0.12 m
3 m	11.1	0.27 m
5 m	6.7	0.75 m
8 m	4.2	1.9 m

At the 0.9 m follow distance the range error is a few centimetres; at 5 m it is most of a metre. The chase controller therefore needs its range only to decide how fast to close in, and steers on bearing, which is accurate at any range.

Depth aligned to colour. The chase pipeline sets the stereo node’s depth alignment to the RGB camera, so each depth pixel is re-projected into the colour camera’s view: back-projected with (11.3) using the right mono camera’s intrinsics, moved by the calibrated mono-to-RGB extrinsics, and projected with the RGB intrinsics. The aligned depth map is 640×400 . This lets a bounding box found in the colour image index the depth map directly.

11.4 SPATIAL DETECTION

The `MobileNetSpatialDetectionNetwork` node combines a detector running on the colour preview (Chapter 12 describes the network) with the aligned depth map. For each detection with normalized box $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$:

1. The box is shrunk about its centre by the *bounding-box scale factor* $s = 0.5$ to a region of

interest

$$\text{ROI} = \left[\bar{x} - \frac{s}{2}w, \bar{x} + \frac{s}{2}w \right] \times \left[\bar{y} - \frac{s}{2}h, \bar{y} + \frac{s}{2}h \right], \quad (11.7)$$

where $(\bar{x}, \bar{y})$ is the box centre and w, h its size. Shrinking keeps the region on the animal's body and off the background around its outline.

2. The depth pixels in the ROI that lie inside the thresholds $[100 \text{ mm}, 8000 \text{ mm}]$ are averaged to give Z . Pixels with no valid disparity (value 0) are excluded.
3. The ROI centre (u, v) and Z are back-projected with (11.3) using the intrinsics of the aligned depth map, giving (X, Y, Z) in millimetres. (DepthAI reports Y positive up , the opposite of (11.1); the chase node uses only X and Z .)

If no pixel in the ROI is valid, $Z = 0$ is reported and the chase node falls back to a bearing from the box position alone.

11.5 IMAGE GEOMETRY OF THE DETECTOR INPUT

The colour camera runs at 1080p (1920×1080) and the network takes a 300×300 *preview*. With DepthAI's default `setPreviewKeepAspectRatio(True)`, the square preview is made by cropping the central 1080×1080 of the frame and scaling it, rather than squashing the full width. The preview then spans a horizontal field of view of

$$\theta_{\text{prev}} = 2 \arctan\left(\frac{1080}{1920} \tan \frac{69^\circ}{2}\right) \approx 42^\circ, \quad (11.8)$$

not the full 69° . (With the calibrated RGB focal length, $f = 1516 \text{ px}$ at 1080p, the full frame spans 64.7° and the crop 39° ; the datasheet and calibration differ by a few degrees.) Two consequences:

- A dog at the edge of the colour frame, outside the central 42° , is not seen by the network at all. The detector's field of view is narrower than the camera's.
- The chase node's original fallback bearing used `rgb_hfov_deg = 68.8` for the preview, which made it about 75% too large. The depth-based bearing $\text{atan2}(X, Z)$ is unaffected because DepthAI maps the ROI back to the aligned depth frame with the correct geometry.

PR #3 sets `setPreviewKeepAspectRatio` explicitly from a `preview_keep_aspect` parameter (default true: centre crop) and computes the preview's field of view at start-up from the RGB intrinsics in the camera's calibration, $\theta_p = 2 \arctan(w/(2f_x))$ with $w = 1080$ for the crop or 1920 for the full width: about 39° with this unit's calibration. Setting `preview_keep_aspect` false gives the network the full width at the cost of a horizontally squashed image.

In flux. Merged in PR #3 but not yet run on the robot.

11.6 CALIBRATION

The camera’s EEPROM held no calibration when it arrived on the robot, and DepthAI refuses to compute depth without one. It was recalibrated on the Windows PC with the depthai v2 calibration tool and a printed ChArUco board, then flashed (2026-10-01). Results:

- Left–right epipolar error: 0.36 px, inside the 0.5 target in `docs/CALIBRATION.md`.
- Baseline: 7.43 cm against 7.5 cm nominal.
- RGB lens position (autofocus step at calibration): 72. The RGB camera must be at the same lens position for the RGB–depth alignment to be exact; autofocus moving the lens shifts the RGB intrinsics slightly.

Table 11.3 lists the calibrated intrinsics. The procedure is in `docs/CALIBRATION.md`, and the calibration file itself is backed up in `camera/pc-tools/`.

Table 11.3: Calibrated intrinsics (from the flashed calibration file)

Camera	Calibrated at	f_x, f_y (px)	(c_x, c_y) (px)	Extrinsic translation
RGB (IMX214)	3840×2160	3032, 3031	(1885, 1099)	reference
Left mono	640×480	450, 450	(335, 238)	$x = -7.43$ cm to right mono
Right mono	640×480	453, 452	(338, 240)	$x = 3.74$ cm to RGB

Intrinsics scale with resolution: at 1080p the RGB focal length is half the 4K value, 1516 px. The PC must use depthai v2: the v3 tools write a calibration format the board’s 2.19 library cannot read.

11.7 MOUNTING AND FRAMES

The camera faces forward on the front of the deck. Its optical frame (`oak_rgb_camera_optical_frame`, Z forward) relates to the robot’s `base_link` (REP-103: x forward, y left, z up) by

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}_{\text{base}} = \begin{bmatrix} Z \\ -X \\ -Y \end{bmatrix}_{\text{cam}} + \mathbf{t}_{\text{cam}}, \quad (11.9)$$

with $\mathbf{t}_{\text{cam}}$ the camera’s mounting offset. The chase node does not use (11.9) today: it steers on the bearing in the camera frame directly, which is equivalent while the camera sits on the robot’s centreline and the offset is small compared with the follow distance.

In flux. No static transform for the camera is published yet. The target filter proposed in §12.7 needs one.

Chapter 12

CHASE: FOLLOWING A DOG OR PERSON

The chase behaviour points the robot at a detected dog (or, for testing, a person) and drives toward it until it is a set distance away. This chapter describes the whole chain: the on-camera pipeline, the neural network that recognises the target, how detections become a track, the control law and its stability, how it shares the motors with teleop, and the plan for bringing it up on the robot. §4.8 gives a short summary of the ROS node.

In flux. Chase has only been run in dry-run mode, where it computes commands but never sends them to the motors. The gyro yaw-rate loop and gyro-stabilized bearing of §12.4.2 are implemented (merged in PR #3) but not yet tested on the robot. It is waiting on the camera's separate power supply (§2.5).

12.1 PIPELINE

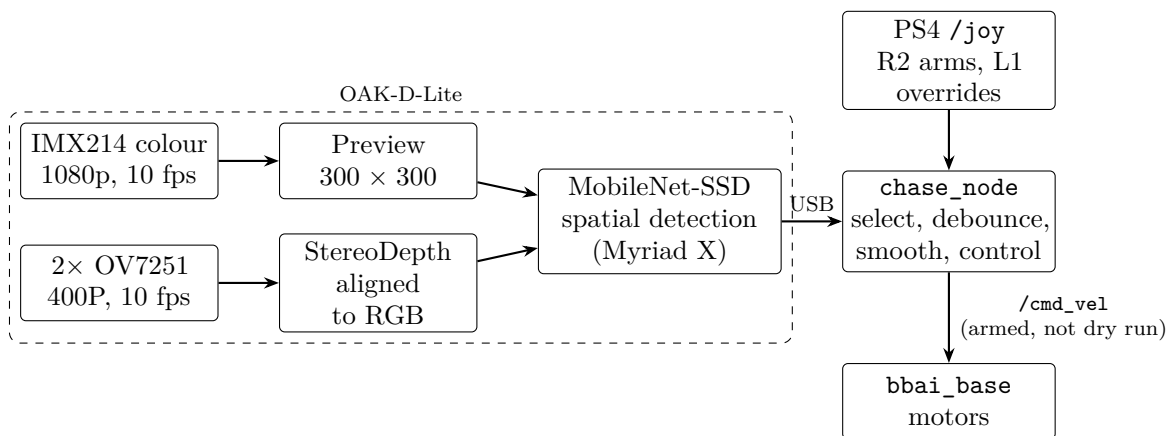


Figure 12.1: Chase pipeline. Everything inside the dashed box runs on the camera; the BeagleBone sees only detections.

Figure 12.1 shows the data flow. The DepthAI pipeline built by the node has five parts: the colour camera at 1080p producing a 300×300 preview, the two mono cameras at 400P, a

StereoDepth node in high-density mode with depth aligned to the colour camera (§11.3), and a **MobileNetSpatialDetectionNetwork** that runs the detector on the preview and attaches a 3-D position to every detection (§11.4). The camera runs at the node’s **fps** parameter, 10 by default, which bounds the detection rate. The model is the OpenVINO 2021.4 MobileNet-SSD compiled for 6 SHAVE cores (`~/models/mobilenet-ssd_openvino_2021.4_6shave.blob`).

A background thread reads detection messages from the camera and keeps the latest list of (label, confidence, X , Z , $\bar{u}$) tuples, with $\bar{u} \in [0, 1]$ the box centre across the image. If the camera throws (USB disconnect, power dip), the thread records the error, reports **CAMERA_ERROR**, and retries every 2 s. (On the board, **numpy** and **rospy** must be imported before **depthai**, or the **depthai** import fails.)

12.2 OBJECT RECOGNITION: MOBILENET-SSD

Backbone. MobileNet [26] replaces each standard convolution with a *depthwise separable* one: a $D_K \times D_K$ convolution applied to each of the M input channels separately, followed by a 1×1 convolution that mixes them into N output channels. On a $D_F \times D_F$ feature map the cost falls from $D_K^2 M N D_F^2$ to $D_K^2 M D_F^2 + M N D_F^2$ multiply-adds, a ratio of

$$\frac{D_K^2 M D_F^2 + M N D_F^2}{D_K^2 M N D_F^2} = \frac{1}{N} + \frac{1}{D_K^2} \approx \frac{1}{9} \quad (D_K = 3, N \gg 9). \quad (12.1)$$

That eight- to nine-fold saving is what lets the detector run at camera frame rate on the Myriad X.

Detector head. SSD [27] predicts objects directly from several feature maps of decreasing resolution (for MobileNet-SSD at 300×300 : 19×19 , 10×10 , 5×5 , 3×3 , 2×2 and 1×1). Each cell of each map carries a few fixed *default boxes* of different sizes and aspect ratios, 1917 in all. For every default box $a = (a_x, a_y, a_w, a_h)$ the network outputs 21 class scores and four offsets (t_x, t_y, t_w, t_h) , decoded to a box by

$$b_x = a_x + \sigma_c t_x a_w, \quad b_y = a_y + \sigma_c t_y a_h, \quad b_w = a_w e^{\sigma_s t_w}, \quad b_h = a_h e^{\sigma_s t_h}, \quad (12.2)$$

with the standard variances $\sigma_c = 0.1$, $\sigma_s = 0.2$. The class scores pass through a softmax. Boxes whose best class score is below the confidence threshold (0.5 here) are dropped, and *non-maximum suppression* removes duplicates: boxes are taken in descending score, and any box overlapping a kept box of the same class by intersection-over-union

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (12.3)$$

above a threshold (typically 0.45) is discarded. All of this runs on the camera.

Classes. The network is trained on PASCAL VOC [28], whose 20 classes plus background include *dog* (index 12) and *person* (index 15), as well as cat, which the network may confuse with a small dog. The node’s **targets** parameter chooses which labels to follow: **[dog]** by default, and **[dog, person]** for testing by walking in front of the camera. In bench tests the network detected the dog at 93–99% confidence, about 10 times per second.

12.3 FROM DETECTIONS TO A TRACK

Each time a new detection frame arrives (about 10 Hz) the node:

1. keeps detections whose label is in **targets** and takes the most confident one;
2. computes the measured bearing and range in the camera frame, using depth if it is valid:

$$(\beta_m, \rho_m) = \begin{cases} (\text{atan2}(X, Z), \sqrt{X^2 + Z^2}) & Z > 0.05 \text{ m}, \\ \left(\arctan[(2\bar{u} - 1) \tan \frac{\theta_p}{2}], \text{unknown} \right) & \text{otherwise,} \end{cases} \quad (12.4)$$

with β positive to the right and θ_p the preview’s horizontal field of view, computed at start-up from the camera’s RGB calibration (§11.5). The fallback is the exact pinhole bearing (11.3), not a linear approximation;

3. converts the bearing to a room-fixed *azimuth* by subtracting the gyro yaw ψ_g at the moment the frame was captured (the detection time minus an assumed 0.15 s camera latency),

$$\alpha_m = \beta_m - \psi_g(t_{\text{det}} - 0.15 \text{ s}), \quad (12.5)$$

so that the robot’s own turning does not change the stored target;

4. smooths both with an exponential moving average (EMA) of weight $a = 0.5$ on the new sample,

$$\hat{\alpha}_k = a\alpha_{m,k} + (1 - a)\hat{\alpha}_{k-1}, \quad (12.6)$$

and the same for ρ (a missing range keeps the previous one);

5. counts consecutive frames with a target, h . A frame with no target resets h to 0, and the next detection restarts the EMA from the raw measurement.

Between frames the current bearing is recovered from the stored azimuth and the latest gyro yaw, $\hat{\beta} = \hat{\alpha} + \psi_g(t)$, which updates at the IMU rate rather than the detection rate. The node publishes the target as a point on **/chase/target** in the camera frame.

The EMA as a filter. With samples every $T = 0.1$ s, (12.6) is the discretization of a first-order low-pass filter with time constant

$$\tau_e = \frac{-T}{\ln(1-a)} = \frac{0.1}{\ln 2} \approx 0.14 \text{ s}, \quad (12.7)$$

which cuts the variance of white bearing noise to a third ($\text{var } \hat{\beta} = \frac{a}{2-a} \text{var } \beta_m = \frac{1}{3} \text{var } \beta_m$) at the cost of 0.14 s of lag.

Debounce and loss. A command is produced only when $h \geq 3$ (three consecutive frames, 0.3 s) and the last detection is less than 0.5 s old. The three-frame rule rejects one-off false positives, but because a single missed frame resets h , it also means one dropped detection stops the robot for at least 0.3 s. Table 12.1 lists the states reported on `/chase/status` twice a second.

Table 12.1: Chase states

State	Meaning
SEARCH	No usable target: none seen, fewer than 3 consecutive hits, or the last one older than 0.5 s. Commands zero.
TRACK	Target held; the status line shows label, confidence, range, bearing and the command.
CAMERA_ERROR	The camera thread failed; it retries every 2 s.
ARMED / idle	Suffix: whether R2 is held (and L1 is not).
(dry run)	Suffix: commands go only to <code>/chase/cmd_vel_preview</code> .

12.4 CONTROL LAW

At 15 Hz the node turns the track into a velocity command:

$$\omega = \begin{cases} \text{sat}_{\omega_{\max}}(-k_{\beta}\hat{\beta}) & |\hat{\beta}| > \beta_{\text{db}}, \\ 0 & \text{otherwise,} \end{cases} \quad (12.8)$$

$$v = \text{sat}_{[0, v_{\max}]}(k_{\rho}(\hat{\rho} - \rho_f)) \cdot \text{sat}_{[0, 1]}\left(1 - \frac{|\hat{\beta}|}{\beta_{\text{full}}}\right) \quad \text{if } \hat{\rho} > \rho_{\text{stop}}, \text{ else } 0, \quad (12.9)$$

with the parameters in Table 12.2. The minus sign in (12.8) is because a target to the right ($\beta > 0$) needs a clockwise turn, which is negative ω in REP-103. The second factor in (12.9) scales forward speed down while the robot is still turning toward the target, reaching zero at 25° off-axis, so the robot turns first and then drives. v is never negative: the robot does not back away from a target that comes too close, it just stops.

12.4.1 Bearing loop

For a stationary target, turning the robot left by $d\psi$ moves the target right by the same angle, so $\dot{\beta} = \omega_{\text{actual}}$. Ignoring saturation and the dead band, and if the robot turns at the commanded rate,

Table 12.2: Chase parameters (`chase.launch`)

Symbol	Parameter	Value	Role
k_β	<code>k_angular</code>	2.5 s^{-1}	turn rate per radian of bearing
$\omega_{\max}$	<code>max_angular</code>	2.0 rad/s	turn-rate limit
β_{db}	<code>bearing_deadband_deg</code>	5°	no turning inside this
β_{full}	<code>full_speed_bearing_deg</code>	25°	forward speed zero beyond this
k_ρ	<code>k_linear</code>	0.4 s^{-1}	forward speed per metre of range error
$v_{\max}$	<code>max_linear</code>	0.2 m/s	forward speed limit (deliberately slow)
ρ_f	<code>follow_distance</code>	0.9 m	range it settles at
ρ_{stop}	<code>stop_distance</code>	0.5 m	no forward motion inside this
a	<code>smoothing</code>	0.5	EMA weight
	<code>min_hits, lost_timeout</code>	$3, 0.5 \text{ s}$	debounce and loss
	<code>rate, fps</code>	$15 \text{ Hz}, 10$	control and camera rates
k_p, k_i	<code>yaw_kp, yaw_ki</code>	$1, 8 \text{ s}^{-1}$	yaw-rate loop gains (PR #3)
ω_c	<code>max_turn_cmd</code>	8 rad/s	turn command limit, duty about 0.47
	<code>min_spin_cmd, gyro_timeout</code>	$4 \text{ rad/s}, 0.3 \text{ s}$	open-loop fallback without gyro
	<code>camera_latency</code>	0.15 s	capture-to-host delay for (12.5)

the loop is

$$\dot{\beta} = -k_\beta \beta, \quad (12.10)$$

which converges exponentially with time constant $1/k_\beta = 0.4 \text{ s}$.

Delay and stability. The real loop has delay: the camera frame period and network latency, the 15 Hz control and 30 Hz base loops, and the motor's response, which together are roughly $\tau_d \approx 0.2 \text{ s}$, plus the EMA's lag $\tau_e = 0.14 \text{ s}$. The open-loop transfer function is then

$$L(s) = \frac{k_\beta e^{-s\tau_d}}{s(\tau_e s + 1)}. \quad (12.11)$$

Its gain crosses one at $\omega_c \approx 2.4 \text{ rad/s}$, where the phase margin is

$$\phi_m = 90^\circ - \frac{180^\circ}{\pi} \omega_c \tau_d - \arctan(\omega_c \tau_e) \approx 90 - 28 - 19 = 44^\circ. \quad (12.12)$$

That is adequate: the step response will overshoot slightly but settle. The loop would go unstable at a total delay of about 0.52 s , and the margin shrinks quickly with gain (Table 12.3), which is why *raising k_β is not the way to make the robot turn harder*.

Table 12.3: Bearing-loop phase margin, $\tau_e = 0.14 \text{ s}$

	$\tau_d = 0.15 \text{ s}$	$\tau_d = 0.2 \text{ s}$	$\tau_d = 0.3 \text{ s}$
$k_\beta = 2.5$	51°	44°	30°
$k_\beta = 4$	32°	22°	2°
$k_\beta = 6$	13°	unstable	unstable

Will the robot actually turn? The bearing loop assumes the robot turns at the commanded rate, and on this robot it does not (§5.6). The detector only sees the central part of the colour frame, about $\pm 20^\circ$ (§11.5), so the largest rate chase requests is about $2.5 \times 0.35 \approx 0.9 \text{ rad/s}$, and even the 2 rad/s limit maps through (5.10)–(5.11) with the geometric track to a duty of only 0.16–0.21. Teleop’s old 1.5 rad/s full stick (duty 0.19) could not start a spin on the floor. Sent open loop, chase turns would stall: the motors form a dead zone in the loop and the bearing error would sit there. Raising k_β is not the answer (Table 12.3). §12.4.2 describes the fix.

12.4.2 Yaw-rate loop and gyro-stabilized bearing

Implemented (PR #3), not yet tested on the robot. The chase node closes a yaw-rate loop of its own on the bias-corrected gyro z rate ω_g from `/imu/data_raw`, leaving `bbai_base` unchanged. The bearing law (12.8) now produces a requested rate ω^* , and the node sends

$$\omega_{\text{cmd}} = \text{sat}_{\omega_c}(\omega^* + k_p(\omega^* - \omega_g) + I), \quad I_{k+1} = I_k + k_i \Delta t (\omega^* - \omega_g), \quad (12.13)$$

with $k_p = 1$, $k_i = 8 \text{ s}^{-1}$ and output limit $\omega_c = 8 \text{ rad/s}$, which by (5.10)–(5.11) is a duty of about 0.47. The feed-forward term passes ω^* straight through; the integrator supplies the extra command a skid-steer turn needs (5.14): while the robot is stuck, $\omega_g \approx 0$ and I grows at $k_i \omega^*$, so a 0.9 rad/s request reaches the limit in about a second. Once the tyres break free, ω_g catches up and I settles at whatever the floor needs. Safeguards:

- *Anti-windup:* I stops integrating while the output is saturated in the direction of the error, and is clamped so it only ever adds effort in the requested direction.
- *Resets:* I is cleared when no turn is requested, when the turn direction flips, and whenever chase is not armed.
- *Gyro loss:* if the gyro is more than 0.3 s old, the loop falls back to open loop with a minimum spin command of 4 rad/s (duty about 0.3).

This is the yaw-rate loop of §6.6, implemented in the chase node rather than the base, so it helps chase only; teleop is unaffected. The duty figures above assume the geometric track. With `cmd_turn_track` = 0.46 m in the base ((5.12)), a 0.9 rad/s request already maps to a duty of about 0.27, and the 8 rad/s limit runs into the base’s 0.6 duty cap; the integrator then has less to do, and its anti-windup keeps it from overshooting.

Effect on the bearing loop. With the azimuth form (12.5), the robot’s own rotation reaches the bearing through the gyro, with a delay of a few tens of milliseconds, instead of through the camera’s 0.2 s. The delay analysis above then applies only to the target’s own motion, and the margin for the robot’s rotation is much larger. The accuracy of the gyro-integrated yaw over the few seconds of a chase is far better than the detector’s bearing noise.

12.4.3 Range loop

For a stationary target straight ahead, $\dot{\rho} = -v$, and (12.9) gives $\dot{\rho} = -k_{\rho}(\rho - \rho_f)$ away from the limits: the range settles at ρ_f with time constant $1/k_{\rho} = 2.5$ s. Beyond $\rho_f + v_{\max}/k_{\rho} = 1.4$ m the speed saturates and the robot closes at a steady 0.2 m/s. A target receding at speed $u < v_{\max}$ is followed with a steady lag $\rho - \rho_f = u/k_{\rho}$ (0.25 m at 0.1 m/s); a target faster than 0.2 m/s, which includes a walking dog, is lost. At these settings the behaviour is a slow *follow*, not a chase, which is the intent for first tests. The range loop needs no delay analysis: its bandwidth is far below the delay.

12.5 ARBITRATION AND SAFETY

- **Dry run by default.** Unless launched with `dry_run:=false`, chase never publishes `/cmd_vel`; it only publishes what it would send on `/chase/cmd_vel_preview`.
- **Dead-man arming.** Commands go to `/cmd_vel` only while R2 (button 7) is held and L1 (button 4, teleop's dead-man) is not. Releasing R2 sends one zero command. Teleop and chase therefore never publish at the same time, and the base node, which applies whichever command arrived last, needs no arbitration logic of its own.
- **Timeouts.** Chase stops if `/joy` is silent for 0.5 s (controller out of range) or the target is lost for 0.5 s; the base node stops the motors if `/cmd_vel` is silent for 0.5 s. Every failure ends in stopped motors.
- **Speed.** 0.2 m/s forward, no reverse, no forward motion closer than 0.5 m.
- **No obstacle avoidance.** Chase drives straight at the target and does not look at the lidar. Anything between the robot and the dog will be hit at up to 0.2 m/s.

12.6 BRING-UP PLAN

Once the camera runs from the battery bank through the USB splitter:

1. **Power soak.** Run `robot.launch`, lidar and chase (dry run) together for several minutes while driving with teleop, and confirm no reset and no `CAMERA_ERROR`.
2. **Dry run with a person.** `dry_run_test.sh` launches chase with `targets:=[dog, person]`, then prints the status, the preview commands and the publishers of `/cmd_vel`, which must not include chase. Walk across the field of view and check that the sign of `angular.z` is positive when you are on the robot's left.
3. **Check the field of view.** Note the bearing at which detections stop at the frame edges; about 20° confirms the cropped preview (§11.5).

4. **Wheels lifted, armed.** Launch with `dry_run:=false`, hold R2, and check that the wheels turn the right way as a person moves.
5. **On the floor with a person.** Check turning on the actual floor (the stall described above), the follow distance and the stop.
6. **With the dog.** Only after the above, with `targets:=[dog]`.

12.7 PROPOSED IMPROVEMENTS

None of these is implemented yet.

Hysteresis on loss. Decrement the hit counter on a missed frame instead of resetting it, and keep commanding the last smoothed bearing until the 0.5 s loss timeout, so a single dropped detection does not stop the robot.

Target filter in the odometry frame. Today the bearing mixes the target’s motion with the robot’s own rotation, and the track is lost the moment the target leaves the image. Tracking the target in `odom` separates the two. With target state $\mathbf{s} = (p_x, p_y, \dot{p}_x, \dot{p}_y)$ and a constant-velocity model,

$$\mathbf{s}_{k+1} = \begin{bmatrix} I & TI \\ 0 & I \end{bmatrix} \mathbf{s}_k + \mathbf{w}_k, \quad \mathbf{w}_k \sim \mathcal{N}\left(0, q \begin{bmatrix} \frac{T^3}{3}I & \frac{T^2}{2}I \\ \frac{T^2}{2}I & TI \end{bmatrix}\right), \quad (12.14)$$

each detection is mapped into `odom` through the camera transform (11.9) and the robot pose (x, y, ψ) ,

$$\mathbf{z}_k = \begin{bmatrix} x \\ y \end{bmatrix} + R(\psi) \left(\begin{bmatrix} Z \\ -X \end{bmatrix} + \mathbf{t}_{\text{cam}} \right), \quad R_z = J \text{diag}(\sigma_\beta^2 \rho^2, \delta Z^2) J^\top, \quad (12.15)$$

where J rotates the bearing/range errors into `odom` and δZ comes from (11.6). A standard Kalman filter (Chapter 9) then gives a target position and velocity that survive short dropouts, can be extrapolated over the detection delay, and can steer the robot back toward where the target left the image.

Search. When the target has been lost for longer than a few seconds, turn slowly in place toward its last known azimuth. The yaw-rate loop of §12.4.2 makes slow turns possible.

Obstacle gating. Limit forward speed from the lidar’s closest return d in a sector ahead of the robot, $v \leq \sqrt{2a_b \max(0, d - d_s)}$, with a braking deceleration a_b and a safety distance d_s , so the robot can always stop before an obstacle. A full alternative is to send the target position as a `move_base` goal once navigation is set up.

Faster detector input. Running the camera at 15–20 fps reduces the delay and allows a higher k_β for the same margin, at the cost of more camera power.

Chapter 13

OPERATING MODES

The software is arranged so that each sensor can be left out and the rest still works. This chapter lists the modes that exist today, what each one estimates, and how to start it.

13.1 LOCALIZATION AND MAPPING MODES

Table 13.1: Localization and mapping modes

Mode	Sensors	Start	Position estimate
A. gmapping SLAM (default)	lidar, encoders, gyro	<code>robot.launch;</code> <code>start_slam.sh</code>	map: odometry corrected by scan matching; drift bounded while the map is consistent.
B. hector SLAM with odometry TF	lidar (odometry only as TF)	<code>start_slam.sh</code> <code>hector</code>	map: scan matching only; fails under fast turns.
C. hector, lidar only	lidar	<code>robot.launch</code> , then <code>start_slam.sh</code> <code>lidar_only</code> (or the A1 alone)	As B, with no wheel odometry at all; for carrying the robot.
D. Dead reckoning	encoders, gyro	<code>robot.launch</code> <code>lidar:=false</code>	odom only: distance from encoders, heading from gyro with ZUPT; unbounded drift (§5.4).
E. Encoder heading	encoders	<code>use_gyro_heading:</code> <code>false</code> in <code>base.yaml</code>	odom with heading from (5.4); turns overestimated by χ (§5.3). Diagnostic only.
F. GPS fusion (parallel to A–D)	GPS, compass, gyro, encoders	<code>gps_fusion.launch</code> on the laptop	gps_map: bounded by GPS when there is a fix; coasts on odometry with compass heading otherwise.
G. Autonomous exploration	lidar, encoders, gyro	<code>robot.launch;</code> <code>explore_house.sh</code> on the laptop	As A; the robot drives itself to unexplored frontiers (§13.4).

Mode F runs alongside any of A–D because it publishes no transform. In practice: indoors, run A and ignore F; outdoors, run F, and A as well if there are walls or obstacles within the lidar’s range.

13.2 WHAT HAPPENS WHEN A SENSOR IS MISSING

Table 13.2: Degradation by sensor

Missing	Effect
GPS fix	Normal indoors. <code>/gps/fix</code> reports <code>NO_FIX</code> ; the EKF coasts. If the first fix comes later, <code>navsat_transform</code> takes its datum then.
Compass	<code>/imu/data</code> not published (<code>mag_heading: false</code> or no field); the EKF has no absolute heading and <code>navsat_transform</code> cannot set a datum, so mode F does not work. Odometry and SLAM are unaffected.
Gyroscope (IMU)	<code>bbai_base</code> cannot start (the IMU is initialised unconditionally). Mode E would be the fallback but needs a code change to skip the IMU.
Encoders	Odometry reports no motion. Mode C still maps. ZUPT would treat the robot as parked whenever no command is active.
One encoder	Its side averages the remaining wheel with a stuck one, so that side reads half its travel; fix the channel list in <code>base.yaml</code> .
Lidar	Modes A–C unavailable; <code>robot.launch lidar:=false</code> gives D.
Motor 1 PRU firmware	<code>/dev/rpmsg_pru32</code> does not appear (the base node logs a warning), <code>rc_motor_init</code> fails, and the motors cannot be driven.
Laptop	Everything on the board runs; no SLAM, no GPS fusion, no RViz.
Bluetooth controller	Motors receive no commands; the 0.5 s timeout keeps them stopped.

13.3 DRIVING MODES

Manual (teleop). Hold L1; left stick drives, right stick turns, R1 adds turbo. Releasing L1 stops the robot.

Chase, dry run. `roslaunch bbai_chase chase.launch`. Holding R2 publishes only the preview topic; nothing moves.

Chase, armed. `chase.launch dry_run:=false`. Holding R2 (with L1 released) lets the chase node drive toward the target. Waiting on separate camera power (§2.5).

Autonomous exploration. `~/bbai_slam/explore_house.sh` on the laptop, with `robot.launch` on the board. The robot maps on its own (§13.4); Ctrl-C stops it and saves the map.

Autonomous (search, map, chase). The default at boot: maps the house once, then patrols it on the saved map and chases a target on sight (Chapter 14). Options on the controller turns it on and off.

Calibration. `roslaunch bbai_base mag_calibrate.py` drives `/cmd_vel` itself to spin two turns. `motor_map_test.py` drives each motor in turn and must be run with `robot.launch` stopped and the wheels lifted.

13.4 AUTONOMOUS EXPLORATION

Mode G lets the robot map the house without a driver. It runs entirely on the laptop on top of gmapping (mode A) and uses three standard ROS components:

gmapping builds the occupancy grid and publishes `map→odom` as in Chapter 10.

explore_lite finds *frontiers*, the boundaries between known free cells and unknown cells [29].

Adjacent frontier cells are grouped, groups smaller than 0.4 m are dropped, and each remaining frontier f is scored by

$$c(f) = \lambda_d d(f) - \lambda_g |f|, \quad (13.1)$$

with $d(f)$ the distance from the robot to the frontier, $|f|$ its size, $\lambda_d = 3$ (**potential_scale**) and $\lambda_g = 1$ (**gain_scale**). The cheapest frontier becomes the next goal, chosen again every 3 s; a goal with no progress for 30 s is blacklisted.

move_base plans and drives to the goal. A global planner (navfn, Dijkstra’s algorithm on the costmap, allowed through unknown space) gives a path, and the DWA local planner [30] follows it at 5 Hz. DWA samples velocity pairs (v, ω) reachable within one control period under the acceleration limits, forward-simulates each for 2 s, and picks

$$(v, \omega)^* = \arg \min_{(v, \omega)} \alpha d_{\text{path}} + \beta d_{\text{goal}} + \gamma c_{\text{obs}}, \quad (13.2)$$

with d_{path} and d_{goal} the end point’s distance from the global path and the goal, c_{obs} the highest costmap cost along the trajectory (trajectories through obstacles are discarded), and $\alpha = 32$, $\beta = 20$, $\gamma = 0.05$.

Costmaps. Obstacles come only from `/scan`, marked up to 3 m and cleared by ray tracing to 6 m. The footprint is 0.22 m×0.20 m plus 2 cm padding, and costs are inflated out to 0.25 m with $c(r) = 252 e^{-8(r-r_i)}$ beyond the inscribed radius r_i . The local costmap is a 3 m×3 m window in `odom`; the global costmap is the SLAM map.

Motion limits. At most 0.2 m/s forward, never backward (the configuration avoids reversing because the lidar is the only obstacle sensor), turn rates 0.6 rad/s to 1.5 rad/s. The 0.6 rad/s minimum exists because slower turns stall the skid steer; with $\chi_c = 0.46$ m it is a duty of about 0.22. Each map is saved to `~/bbai_slam/maps/` with a timestamp when the script is stopped.

Limits and safety.

- The RPLIDAR scans one horizontal plane about 10 cm up. It cannot see stairs, drop-offs or anything lower than the scan plane (shoes, cables, pet bowls), and it may see table and chair legs only as thin points. *Keep the robot away from stairs, or block them, while exploring.*

- The camera stays off during exploration because of the power resets (§2.5), so there is no visual obstacle or dog detection.
- The deadman does not apply: `move_base` publishes `/cmd_vel` itself at 5 Hz. Holding L1 lets teleop interleave its commands, but the two will fight; Ctrl-C on the laptop is the stop, after which the base node's 0.5 s timeout halts the motors.
- Exploration inherits every SLAM weakness of Chapter 10: if the map breaks, the frontiers are wrong and the robot drives toward them.

In flux. Exploration (configuration in `laptop/bbai_slam/nav/`) has not yet been run through a whole house.

Chapter 14

AUTONOMOUS MODE: SEARCH, MAP AND CHASE

The goal of the project is a robot that, once switched on, finds the dog (or the kids) on its own and chases her. Autonomous mode puts the pieces of the previous chapters together: lidar mapping and exploration (§13.4), localization in a saved map, and the chase controller of Chapter 12. It is the robot's default behaviour at boot. The code is the `bbai_autonomy` package (`ros/bbai_autonomy/`).

14.1 BEHAVIOUR

Explore (first run, no saved map). `gmapping` builds the map and `explore_lite` drives frontier to frontier, exactly as in §13.4. When `explore_lite` has sent no new goal for 60 s (and at least 120 s have passed), the house is mapped. The mission saves it as `maps/house_<date>.{pgm,yaml}`, points `maps/house.yaml` at it and restarts the host side in patrol mode. A partial map is also saved every 2 min in case the power fails.

Patrol (every run after that). `map_server` loads `house.yaml` and AMCL localizes the robot in it. The mission spreads search points over the map (§14.5), visits them nearest first, and at each one turns twice by 120° so the camera's 39° view sweeps the whole room. When every point has been visited it starts the next round, in the opposite order, forever.

Chase (any time). The chase node runs in *auto* arm mode: it drives toward a target whenever it holds a track, without R2. While `/chase/active` is true the mission cancels its patrol goal; once the target has been gone for 3 s it re-sends the same goal. During exploration `move_base` simply keeps planning in the background and continues after the chase.

14.2 ARCHITECTURE

The work is split by where it has to run. Anything that touches the motors or the camera stays on the board; anything heavy runs on an off-board *host*, which is either the Proxmox server VM or the

laptop. The host is whatever machine sources `~/bbai_slam/env.sh`; nothing in the package names one.

Table 14.1: Autonomous-mode nodes.

Machine	Nodes	Started by
Board (Melodic)	<code>robot.launch</code> , PS4 teleop, <code>chase</code> (auto), <code>cmd_mux</code>	<code>bbai-robot.service</code>
Host (Noetic)	<code>gmapping</code> + <code>explore_lite</code> <i>or</i> <code>map_server</code> + AMCL; <code>move_base</code> ; <code>mission</code>	<code>bbai-host@<user>.service</code>

Who drives. Three sources want `/cmd_vel`. Rather than let them fight (the problem noted for exploration in §13.4), each publishes to its own topic and `cmd_mux` on the board forwards one of them, in strict priority:

1. PS4 teleop while L1 is held (the mux goes silent; teleop publishes `/cmd_vel` itself);
2. `chase`, while `/chase/cmd_vel` is fresher than 0.4 s;
3. navigation, `/nav/cmd_vel` from `move_base`, fresher than 0.6 s.

Nothing passes while autonomy is disabled. The Options button toggles it, as does `/autonomy/enable`. Each hand-over to nobody sends one zero twist.

Failure behaviour. If the host crashes or the Wi-Fi drops, `/nav/cmd_vel` stops and the base node’s 0.5 s timeout halts the motors. The board still teleops and still chases, because `chase` never depended on the host. When the board reboots it starts a new roscore; the host script watches the master’s `run_id` and restarts its side, so the two re-pair without anyone logging in.

14.3 TARGETS AND THE KID CLASS

Targets are chosen at launch (`targets:=[dog,kid]`) or live with `/chase/set_targets`. Any MobileNet-SSD label works (§12.2), plus `kid` and `adult`.

MobileNet-SSD has a *person* class but no child class, so a person is sorted by the height of the top of their head above the floor. With the box’s top edge at normalized image row $y_{\min}$ (0 at the top), depth Z from the spatial detector, RGB focal length f_y at 1080 p, camera height h_c and upward pitch θ_c ,

$$h_{\text{head}} = h_c + Z \tan\left(\arctan \frac{(0.5 - y_{\min}) 1080}{f_y} + \theta_c\right). \quad (14.1)$$

A person with $h_{\text{head}} < 1.35$ m is a kid, otherwise an adult. Only the top of the box is used, so the feet need not be in view. That matters with the camera this low: with $h_c \approx 0.15$ m and a vertical field of view near 40°, a 1.1 m child fits in the frame head to toe only beyond about 2.6 m.

If the box touches the top of the frame the head is out of view, and (14.1) at $y_{\min} = 0$ is only a lower bound. Above 1.35 m that still proves an adult; below it the node can’t tell and does not start a chase. Once it is already following a kid, such views keep the track, since a kid up close always

fills the frame. The status line prints the estimate (`kid(1.12m tall)`) so h_c , θ_c and the threshold can be tuned with real people in view. The low camera works in our favour here: an adult's head is usually out of frame and above the threshold at the frame top, while a child's head is in view from a couple of metres away.

14.4 SAVED MAP OR A FRESH MAP EACH TIME

The robot maps once and localizes after that. A fresh map each run was rejected for three reasons:

- AMCL on a fixed map is lighter and steadier than gmapping, and a long chase around one room cannot smear the map.
- A finished map gives search points over every room. Frontier exploration stops as soon as no frontiers are left, which is exactly when a search needs to continue.
- Mapping with the RPLIDAR A1 is slow and sometimes fails (Chapter 10). Doing it once and checking the result beats redoing it on every boot.

The cost is that AMCL needs a starting guess. The mission writes the robot's pose to `maps/last_pose.yaml` every 5 s and uses it at the next start, so a robot switched off where it stopped resumes correctly. If it was carried elsewhere, the command `home` declares it at the map origin (where mapping began) and `global` spreads AMCL's particles over the whole map. If the furniture moves, `remap` forgets the map and the next run explores again.

AMCL uses the likelihood-field laser model with 90 of the 360 beams, 300 to 2000 particles (cheap on the host), an update every 0.1 m or 0.2 rad, and a raised rotation noise ($\alpha_1 = 0.2$) for the skid steer's slip when turning.

14.5 SEARCH POINTS

From the occupancy grid M with resolution $r = 0.05$ m:

1. Mark cells free when $0 \leq M < 25$; unknown and occupied cells are not.
2. Erode the free set by $k = \lceil 0.30/r \rceil$ cells (8-connected), so every remaining cell is at least 0.30 m from a wall or unknown space.
3. Flood-fill from the cell nearest the robot. Only reachable cells count, which drops closed rooms and space seen through doorways that are too narrow.
4. Cut the map into $1.5 \text{ m} \times 1.5 \text{ m}$ blocks and keep, in each block that has a reachable cell, the one nearest the block centre.
5. Order the points as a nearest-neighbour tour from the robot.

A goal that fails or takes longer than 90 s is skipped; the next round tries it again.

14.6 RUNNING IT

By hand. Board: `robot_start.sh autostart:=false`. Host: `host_autonomy.sh`. Press Options to start and again to stop.

At boot. Enable `bbai-robot.service` on the board and `bbai-host@<user>` on the host. Boot options go in `/etc/default/bbai-robot`, for example `BBAI_ARGS="targets=[dog,kid] autostart:=false"`; booting with autonomy off and starting it with Options is the safer default until it has run a few times.

Watching. `/autonomy/state` says what the mission is doing (PATROL round 2 point 5/14) and `/autonomy/source` who is driving.

The exact commands are in `docs/COMMANDS.md`.

Limits. Everything listed for exploration in §13.4 still applies, stairs above all. The chase controller has no obstacle avoidance of its own: it drives straight at the target through anything the camera does not report. Its speed cap (0.2 m/s) and stop distance (0.5 m) are the protection.

In flux. Autonomous mode is written and unit-tested off the robot (mux priorities, kid classification, search points on a synthetic map). It has not yet run on the robot.

Chapter 15

KNOWN ISSUES AND NEXT STEPS

15.1 OPEN ISSUES

Board resets under full load. The cape's 5 V rail drops out with the camera, motors, lidar and Bluetooth all running (§2.5). Waiting on a USB splitter and battery bank to power the camera separately.

SLAM loses track while driving. Analysis in §10.5. The parked drift is fixed, and the lidar was found mounted backward; its transform now has a 180° yaw. Longer mapping runs are still to be tested.

Skid-steer turning. Low turn commands cannot overcome scrubbing friction (§5.6). The teleop curve is a workaround; closed-loop wheel-speed control would fix it properly.

Bluetooth input stalls. Controller reports pause for 100–500 ms in bursts. ROS itself was measured clean (no `/cmd_vel` gap over 58 ms). Parked as acceptable for now.

Placeholder static transforms. The lidar, IMU and GPS offsets in `robot.launch` have not been measured.

Constant odometry covariance. `/odom` reports a fixed pose covariance (§5.4). Harmless for the current EKF, which fuses only v_x , but wrong for any consumer of the pose.

Repository behind the robot. The teleop turn curve is only on the board.

Chase turning and field of view. Open-loop chase turns asked for only about 0.2 duty, and the fallback bearing assumed a 69° view where the detector sees about 39°. PR #3 (merged) adds a gyro yaw-rate loop and computes the view from the calibration (§12.4.2, §11.5); not yet tested on the robot.

Chase stops on one missed frame. A single missed detection resets the hit counter (§12.3).

No camera transform. No static transform for the OAK-D-Lite is published.

Pause and Mode buttons. The cape's buttons still do not work.

15.2 SUGGESTED IMPROVEMENTS

In rough order of value for the current goals:

1. Measure the lidar's position on the robot and enter it in `base_link`→`laser`.
2. Map with gentle turns, under about 0.5 rad/s (§10.4).
3. Close the speed loop and add a gyro yaw-rate loop (§6.5–6.6). This makes turning independent of the floor and gives the chase controller the turn rate it asks for.
4. Bring chase up in the order of §12.6 once the camera has its own power, then add the target filter of §12.7.
5. Log `/imu/data_raw` at rest for an hour and fit the error model (7.1)–(7.2) with Allan variance, as in the thesis (thesis §7.5). That gives principled process noise for the EKF and a better ZUPT time constant.
6. Publish a growing pose covariance from (5.9) instead of the constant one.
7. Fuse $v_y = 0$ in the EKF and set a Mahalanobis rejection threshold on GPS (§9.3).
8. Consider `slam_toolbox`, which handles loop closure and lifelong mapping better than `gmapping` and can switch to localisation on a saved map.
9. Align `map` and `gps_map` with (9.8) once there is outdoor data.

Appendix A

REFERENCE

A.1 REPOSITORY MAP

Table A.1: Where things are in `jstumfoll2/bbai-robot`

Path	Contents
<code>devicetree/</code>	Robotics Cape device tree for the BB-AI, patch and full sources
<code>librobotcontrol/</code>	Submodule: the <code>v1.1_AIfixes</code> fork
<code>pru/</code>	Notes on the PRU2_0 <code>gpio_rpmsg</code> firmware
<code>system/</code>	<code>uEnv.txt</code> , <code>btsdio</code> blacklist, encoder tick logger
<code>gps/</code>	<code>libgps</code> patch for <code>/dev/ttyS2</code> at 4800 baud
<code>ros/bbai_base/</code>	Base node, GPS node, compass calibration, <code>base.yaml</code> , <code>robot.launch</code>
<code>ros/bbai_teleop/</code>	PS4 teleop node, Bluetooth bring-up and pairing scripts, <code>systemd</code> unit
<code>ros/bbai_chase/</code>	Chase-the-dog node
<code>laptop/bbai_slam/</code>	<code>gmapping</code> , <code>hector</code> and GPS fusion launch files, EKF config, <code>RViz</code> , scripts
<code>camera/</code>	OAK-D-Lite board scripts, PC calibration tools, calibration backup
<code>docs/</code>	Goals, handoff, calibration guide, command cheat sheet, this document

The day-to-day commands for the board, laptop, PC and controller are in `docs/COMMANDS.md`.

A.2 BASE NODE PARAMETERS

A.3 SYMBOLS

A.4 BUILDING THIS DOCUMENT

```
cd docs/robot-system
make # pdflatex + bibtex; or: latexmk -pdf main.tex
```

Table A.2: base.yaml

Key	Value	Meaning
left_encoders, right_encoders	[1, 3], [2, 4]	encoder channels per side
encoder_signs	{1:1, 2:-1, 3:1, 4:-1}	forward counts up
counts_per_rev	15.0	from the 3 ft push
wheel_radius	0.0396	m
track_width	0.120	m; geometric track, used for encoder heading
cmd_turn_track	0.46	m; effective track for /cmd_vel turns (5.12)
use_gyro_heading	true	heading from the gyroscope
publish_tf	true	publish odom→base_link
rate	30.0	Hz
left_motors, right_motors	[1, 2], [3, 4]	motor channels per side
motor_signs	all 1	positive duty drives forward
max_wheel_speed	1.2	m/s at duty 1 (lifted)
max_duty / min_duty	0.6 / 0.12	duty cap and deadzone floor
cmd_timeout	0.5	s
zupt_delay / zupt_bias_tau	0.5 / 20.0	s; defaults in code
mag_heading	true	publish /imu/data
mag_offset_x / _y	11.71 / 66.61	μ T, hard-iron offset
mag_sign	1	compass direction
declination_deg	2.5	east positive

Table A.3: Main symbols

x, y, ψ	planar pose; ψ is ENU yaw, counter-clockwise from east
v, ω	body speed and yaw rate
r, b, L	wheel radius, track, wheelbase
χ	skid-steer effective-track factor
δ	wheel travel per encoder count
$g_z, \hat{b}_g$	gyroscope z rate and its bias estimate
$\mathbf{m}, \mathbf{o}$	magnetometer reading and hard-iron offset
D	magnetic declination, east positive
χ_c	compass heading, clockwise from north ($= \pi/2 - \psi$)
φ, λ, h	latitude, longitude, ellipsoidal height
P, Q, R, K, H	EKF covariance, process noise, measurement noise, gain, measurement matrix
ℓ_i	occupancy log-odds of cell i
M, N_{eff}	particle count and effective sample size

It needs a TeX distribution with the `amsmath`, `siunitx`, `booktabs`, `tikz` and `listings` packages (on Ubuntu: `texlive-latex-extra texlive-science`).

Bibliography

- [1] J. M. Stumfoll, “Discrete-time neural network based state observer with neural network based control formulation for a class of systems with unmatched uncertainties,” m.s. thesis, revision 8, Missouri University of Science and Technology, n.d. Source: <https://github.com/jstumfoll2/twip-dmso>, docs/thesis.
- [2] T. Foote and M. Purvis, “REP 103: Standard units of measure and coordinate conventions.” <https://www.ros.org/reps/rep-0103.html>, 2010.
- [3] W. Meeussen, “REP 105: Coordinate frames for mobile platforms.” <https://www.ros.org/reps/rep-0105.html>, 2010.
- [4] A. Mandow, J. L. Martínez, J. Morales, J. L. Blanco, A. García-Cerezo, and J. González, “Experimental kinematics for wheeled skid-steer mobile robots,” *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 1222–1227, 2007.
- [5] J. L. Martínez, A. Mandow, J. Morales, S. Pedraza, and A. García-Cerezo, “Approximating kinematics for tracked mobile robots,” *The International Journal of Robotics Research*, vol. 24, no. 10, pp. 867–878, 2005.
- [6] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [7] J. Y. Wong, *Theory of Ground Vehicles*. Hoboken, NJ: John Wiley and Sons, 4th ed., 2008.
- [8] D. Gebre-Egziabher, *Design and Performance Analysis of a Low-Cost Aided Dead Reckoning Navigator*. PhD thesis, Stanford University, 2004.
- [9] W. S. Flenniken, “Modeling inertial measurement units and analyzing the effect of their errors in navigation applications,” Master’s thesis, Auburn University, 2005.
- [10] P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*. Boston, MA: Artech House, 2nd ed., 2013.
- [11] J. B. Kuipers, *Quaternions and Rotation Sequences*. Princeton, NJ: Princeton University Press, 1999.

- [12] D. Gebre-Egziabher, G. H. Elkaim, J. D. Powell, and B. W. Parkinson, "Calibration of strapdown magnetometers in magnetic field domain," *Journal of Aerospace Engineering*, vol. 19, no. 2, pp. 87–102, 2006.
- [13] I. Kåsa, "A circle fitting procedure and its error analysis," *IEEE Transactions on Instrumentation and Measurement*, vol. IM-25, no. 1, pp. 8–14, 1976.
- [14] NOAA National Centers for Environmental Information, "The world magnetic model, WMM2025." <https://www.ncei.noaa.gov/products/world-magnetic-model>, 2025.
- [15] W. T. Higgins, "A comparison of complementary and Kalman filtering," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 11, no. 3, pp. 321–325, 1975.
- [16] P. Misra and P. Enge, *Global Positioning System: Signals, Measurements, and Performance*. Lincoln, MA: Ganga-Jamuna Press, 2nd ed., 2006.
- [17] T. Moore and D. Stouch, "A generalized extended Kalman filter implementation for the Robot Operating System," in *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS-13)*, pp. 335–348, Springer, 2014.
- [18] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Transactions of the ASME, Journal of Basic Engineering*, vol. 82, pp. 35–45, 1960.
- [19] D. Simon, *Optimal State Estimation*. Hoboken, NJ: John Wiley and Sons, 2006.
- [20] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [21] G. Grisetti, C. Stachniss, and W. Burgard, "Improved techniques for grid mapping with Rao-Blackwellized particle filters," *IEEE Transactions on Robotics*, vol. 23, no. 1, pp. 34–46, 2007.
- [22] A. Doucet, N. de Freitas, K. Murphy, and S. Russell, "Rao-Blackwellised particle filtering for dynamic Bayesian networks," in *Proceedings of the Sixteenth Conference on Uncertainty in Artificial Intelligence*, pp. 176–183, 2000.
- [23] K. Murphy, "Bayesian map learning in dynamic environments," in *Advances in Neural Information Processing Systems 12*, pp. 1015–1021, 1999.
- [24] S. Kohlbrecher, O. von Stryk, J. Meyer, and U. Klingauf, "A flexible and scalable SLAM system with full 3D motion estimation," in *IEEE International Symposium on Safety, Security, and Rescue Robotics*, pp. 155–160, 2011.
- [25] H. Hirschmüller, "Stereo processing by semiglobal matching and mutual information," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008.

- [26] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXiv preprint arXiv:1704.04861*, 2017.
- [27] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “SSD: Single shot multibox detector,” in *European Conference on Computer Vision (ECCV)*, pp. 21–37, 2016.
- [28] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL visual object classes (VOC) challenge,” *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.
- [29] B. Yamauchi, “A frontier-based approach for autonomous exploration,” in *Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA)*, pp. 146–151, 1997.
- [30] D. Fox, W. Burgard, and S. Thrun, “The dynamic window approach to collision avoidance,” *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.